

Olingo: Threshold Lattice Signatures with DKG and Identifiable Abort

Kamil Doruk Gur

University of Maryland
dgur1@proton.me

Patrick Hough

Universität der Bundeswehr München
patrick.hough@unibw.de

Jonathan Katz

Google
jkatz2@gmail.com

Caroline Sandsbråten

Norwegian University of
Science and Technology
caroline.sandsbraten@ntnu.no

Tjerand Silde

Norwegian University of
Science and Technology
tjerand.silde@ntnu.no

Abstract

We present Olingo, a framework for threshold lattice signatures that is the first to offer all desired properties for real-world implementations of quantum-secure threshold signatures: small keys and signatures, low communication and round complexity, non-interactive online signing, distributed key generation (DKG), and identifiable abort.

Our starting point is the framework by Gur, Katz, and Silde (PQCrypto 2024). We change the underlying signature scheme to Raccoon (Katsumata et al, Crypto 2024), remove the trapdoor commitments, extend the scheme from two to three rounds, and then apply numerous improvements and optimizations to achieve all the above properties. We provide detailed proofs of security for our new framework and present concrete parameters and benchmarks.

At the 128-bit security level, for up to 1024 parties and supporting 2^{60} signatures, our scheme has 4.4 KB public keys and 11.3 KB signatures; while signing requires communication of 852 KB per party using the LaBRADOR proof system (Beullens and Seiler, Crypto 2023). An optimistic non-interactive version of our scheme requires only 76 KB communication per party.¹

Keywords

Threshold Signatures, Lattices, DKG, Identifiable Abort

1 Introduction

In a t -out-of- n threshold scheme [32, 33], a key is split into n shares such that t of those shares are needed to perform some cryptographic operation. Threshold schemes ensure both improved security and resilience: an adversary who corrupts fewer than t parties cannot carry out the cryptographic operation on its own; at the same time, even if $n - t$ parties are unavailable, the remaining t parties can still perform the desired computation.

Threshold signatures have seen increased interest in recent years (e.g., [6, 34, 54]) due, in part, to their application to blockchain wallets. Indicative of this interest, NIST recently issued a call for multi-party threshold schemes [21]. Most existing threshold signatures are, however, based on classical hardness assumptions, leaving them vulnerable to cryptographically relevant quantum computers. This

has recently motivated works on (plausibly) *quantum-secure* threshold signatures, especially constructions based on lattice assumptions [17, 24, 30, 37, 48, 51, 66, 67, 75]. For practical deployments, however, such schemes should satisfy the following requirements:

- Security based on standard hardness assumptions.
- Scalability to a large number of users n (NIST calls for up to $n = 1024$) with flexible choice of threshold t .
- Key and signature sizes comparable to those of standardized (non-threshold) signature schemes.
- Support for *non-interactive online signing* so that, following an offline, message-independent, pre-processing phase, parties can sign a message in only one round of online communication. The total communication and round complexity should also be low.
- Distributed key generation (DKG) to avoid trusted setup.
- Identifiable abort so a misbehaving party can be identified in case signature generation fails.

Recent work on lattice-based threshold signatures shows schemes satisfying some—but not all—of the above requirements. (See Table 1.) The most efficient schemes [17, 24, 30, 37, 51, 66, 67, 75] lack DKG or identifiable abort (usually both), while the only scheme offering both² [48] does not have non-interactive online signing and is significantly less efficient.

1.1 Our Contributions

We propose a new framework for lattice-based threshold signatures called Olingo.³ When applied to the Raccoon signature scheme [31], it is concretely efficient and offers DKG and identifiable aborts based on standard lattice assumptions. In more detail:

- Our public keys and signatures have size 4.4 KB and 11.3 KB, respectively, comparable to the original Raccoon scheme [31] and the most efficient lattice-based threshold schemes [17, 30, 37, 51, 66, 67]. In particular, we achieve the smallest signature sizes of all lattice-based designs as well as the smallest combined size of public key and signature.

¹The only changes in the abstract compared to at the submission server are minor improvements and clarifications regarding the concrete parameters.

²Although the scheme allows for identifying misbehaving parties, the authors do not claim (and no proof has been given) that the scheme satisfies the formal requirements of identifiable abort.

³The Olingo is a mammal of the Procyonidae family, which among others includes the Raccoon. Olingos are not masked and are thus *identifiable*.

Table 1: Comparison of lattice-based threshold signature schemes. COM is total communication; all sizes are in KB. “—” means the data is not provided by the authors. The number of rounds is given as “offline + online rounds,” so “ $X + 1$ ” rounds means that the scheme has non-interactive online signing. ID-A means identifiable abort. S means “sequential” and C means “concurrent” for the security model.

Scheme	PK	SIG	COM	Rounds	Users	DKG	ID-A	Model
[48]	13.6	46.6	3000	0 + 2	5	✓	(✓) ⁶	C
[30]	3.9	12.7	28	0 + 3	1024	✗	✗	C
[37]	5.5	10.8	538	1 + 1	1024	✗	✗	S
[37, 75]	8.7	30.9	767	1 + 1	1024	✗	✗	S
[51]	3.9	12.7	—	2 + 3	1024	✗	✗	S
[24, 75]	42.1	144.5	1240	1 + 1	32	✗	✗	S
[17]	4.5	13.4	629	1 + 1	1024	✗	✗	C
[67]	2.6	2.7	5.6	2 + 1	8	✗	✗	S
[66]	3.9	12.7	$88 + 6.4t$	0 + 3	1024	✗	✓	C
[66] (Optimistic)	3.9	12.7	28	0 + 3	1024	✗	✓	C
Olingo	4.4	11.3	852	2 + 1 (0 + 3)	1024	✓	✓	S (C)
Olingo (Optimistic)	4.4	11.3	76	2 + 1 (0 + 3)	1024	✓	✓	S (C)

- Our signing protocol has three rounds of communication, and offers non-interactive online signing.⁴
- Generating a signature requires 852 KB total (142 KB online) communication when using the LaBRADOR proof system [12]. An optimistic version of our scheme only requires 76 KB of communication in the last round.
- Our scheme is compatible with an efficient, 2-round DKG protocol that we describe as part of this work. Our DKG protocol relies on a new scheme for verifiable secret sharing in the lattice setting that we believe is of independent interest.
- Our signing protocol (though not our DKG protocol) has identifiable aborts, so misbehaving parties can be identified and then rebooted and/or excluded from further executions.
- Our implementation of the scheme shows that it is practical, with signing taking $(3.1 + 0.7 \cdot t)$ seconds total⁵ (only 1.4 seconds for the non-interactive online phase).

A drawback of Olingo is that it is only proven secure in a sequential setting, in contrast to some more-recent schemes [17, 30, 48, 66] that offer concurrent security. However, our protocol can be made concurrently secure if we sacrifice online signing.

We compare Olingo to prior work in Table 1 and detail the parameters and benchmarks in Section 7. We also provide a more in-depth discussion of related work in Appendix A. We include two versions of Olingo: the complete communication and an optimistic version which is more efficient when all parties are honest. For completeness we also include an optimistic version of [66].

⁴Only the final round of our protocol depends on the message being signed. Our security definition, however, requires that messages being signed are independent of the offline phase. In practical deployments we believe this level of security suffices.

⁵Note that the LaZer library [60] does not yet support larger parameter sets, so our benchmarks are conducted with the LNP [59] proof system instead.

⁶The scheme allows for identifying aborting parties, but no proof that it satisfies identifiable abort is given.

1.2 Technical Overview

Our starting point is the work of Gur, Katz, and Silde [48] (the GKS-framework). Their framework is compatible with a DKG protocol and identifiable aborts, but the concrete efficiency is poor and it does not have non-interactive online signing. At a high level, we add a round to their overall signing protocol, and observe that this allows for both improving performance and achieving non-interactive online signing.

The base scheme of our construction is Raccoon [31]. Fix a ring R_q . The prover’s private key is a short vector $\mathbf{s} \in R_q^{\ell+k}$, and its public key consists of a matrix $\bar{\mathbf{A}} := [\mathbf{A} \mid \mathbf{I}] \in R_q^{k \times (\ell+k)}$ (where $\mathbf{A}$ is uniform) and the vector $\mathbf{y} := \bar{\mathbf{A}}\mathbf{s}$. Execution of the signature scheme proceeds as follows (simplified):

- (1) Prover samples bounded $\mathbf{r} \in R_q^{\ell+k}$ and sends $\mathbf{w} := \bar{\mathbf{A}}\mathbf{r}$.
- (2) Verifier responds with a short challenge $c \in R_q$.
- (3) Prover responds with bounded vector $\mathbf{z} := c \cdot \mathbf{s} + \mathbf{r}$.
- (4) Verifier accepts iff $\mathbf{z}$ is bounded and $\bar{\mathbf{A}}\mathbf{z} = c \cdot \mathbf{y} + \mathbf{w}$.

To overcome the challenges with large Shamir secret sharing coefficients, the GKS-framework uses a linearly homomorphic encryption scheme to compute the linear operations above in the encrypted domain. Instead of sharing $\mathbf{s}$ itself, the signers each hold an encryption $\text{ctx}_{\mathbf{s}} = \text{Enc}(\mathbf{s})$ of $\mathbf{s}$ with respect to a known public encryption key pk , and share the corresponding decryption key sk in a t -out-of- n fashion. Any set $\mathcal{U} \subseteq [n]$ of t parties can generate a signature as follows:

- (1) Party i samples a bounded $\mathbf{r}_i \in R_q^{\ell+k}$, and computes:
 - $\mathbf{w}_i := \bar{\mathbf{A}}\mathbf{r}_i$ (the first message in the signature)
 - $\text{ctx}_{\mathbf{r}_i} = \text{Enc}(\mathbf{r}_i)$ (homomorphic encryption of $\mathbf{r}_i$)
 - $\text{com}_i = \text{Com}(\mathbf{w}_i)$ (homomorphic com. of $\mathbf{w}_i$)
 Party i broadcasts $\text{ctx}_{\mathbf{r}_i}$ and com_i to all other parties.
- (2) Each party in $\mathcal{U}$ locally computes $\text{com} := \sum_{i \in \mathcal{U}} \text{com}_i$, the challenge $c := H(\mathbf{w})$, and an “encrypted (partial) signature” $\text{ctx}_{\mathbf{z}} := c \cdot \text{ctx}_{\mathbf{s}} + \sum_{i \in \mathcal{U}} \text{ctx}_{\mathbf{r}_i}$. The i th party then broadcast its threshold decryption share of $\text{ctx}_{\mathbf{z}}$.
- (3) Given decryption shares from all parties, they decrypt $\text{ctx}_{\mathbf{z}}$ to obtain $\mathbf{z}$, compress it, and outputs signature $(c, \mathbf{z})$.

We also need to send the opening of the commitments in the last round of the GKS framework, and the sum of these openings is required as a part of the signature to be able to verify it. This both increases the signature size and requires the verification algorithm to change. Furthermore, the commitment key must be message-dependent to avoid various attacks [28, 35], so the message must be fixed at the beginning for the parties to start the signing process. Finally, zero-knowledge proofs must be added for active security.

On a high level, removing the trapdoor commitments allows us to use a standard (non-rejecting) base signature scheme [31] with essentially the original parameters, and apply state-of-the-art compression techniques [7, 30, 58] before outputting the signatures. This also makes our scheme non-interactive, since the first round commitment key dependency in GKS is removed. We furthermore replace statistical noise-drowning techniques with improved noise analysis based on the MLWE with Hints assumption [53] and Rényi Divergence [8] to optimize the performance before we instantiate our scheme with concrete parameters and provide benchmarks. We also optimize the distributed key generation protocol and signature

procedure in GKS by carefully analyzing how to combine efficient secret sharing [23, 71] with state-of-the-art zero-knowledge proofs for lattices [9, 12, 59]. This results in major changes to GKS, and we provide new and complete proofs of security for all parts of our improved framework. This gives us Olingo, our new and efficient three-round framework:

- (1) For $i \in \mathcal{U}$, the i th party samples a bounded $\mathbf{r}_i \in R_q^{\ell+k}$, computes $\mathbf{w}_i := \tilde{\mathbf{A}}\mathbf{r}_i$, and broadcast a hash $h_i = H(\mathbf{w}_i)$.
- (2) When all hashes are received, compute a homomorphic encryption $\text{ctx}_{\mathbf{r}_i} = \text{Enc}(\mathbf{r}_i)$, and broadcast $\mathbf{w}_i$ and $\text{ctx}_{\mathbf{r}_i}$.
- (3) Each party in $\mathcal{U}$ locally computes $\mathbf{w} := \sum_{i \in \mathcal{U}} \mathbf{w}_i$, the challenge $c := H(\mathbf{w})$, and an “encrypted (partial) signature” $\text{ctx}_{\mathbf{z}} := c \cdot \text{ctx}_{\mathbf{s}} + \sum_{i \in \mathcal{U}} \text{ctx}_{\mathbf{r}_i}$. The i th party then broadcast its threshold decryption share of $\text{ctx}_{\mathbf{z}}$.
- (4) Given decryption shares from all parties in $\mathcal{U}$, each signer can decrypt $\text{ctx}_{\mathbf{z}}$ to $\mathbf{z}$, and output signature $(c, \mathbf{z})$.

Note that we also need to add zero-knowledge proofs to ensure that everything is computed correctly and we had to add one additional round of communication compared to GKS. That said, Olingo allows for many optimization not possible earlier, leading to a much more efficient scheme in practice, as shown in in Table 1.

1.3 Independent and Concurrent Work

Three independent and concurrent works were recently posted online. del Pino et al. [29] present an efficient threshold signature scheme from lattices with both a DKG and identifiable aborts based on new secret sharing techniques. Bienstock et al. [13] showcase efficient MPC techniques to compute pure ML-DSA [58] signatures in the honest majority setting. Borin et al. [16] design and implement efficient threshold versions of both TRaccoon (with a DKG and identifiable aborts) and ML-DSA for 64 and 6 users, respectively, by optimizing prior works discussed above.

2 Background

2.1 Communication Model

We assume a synchronous communication model with private and authenticated links between each pair of parties and a broadcast channel [39, 44]. The synchronous property means that protocols proceed in a sequence of rounds. In each round, a party performs some local computation, sends messages (if any) through the private and authenticated links, and broadcasts some information over the broadcast channel. By the end of the round, it also receives all messages sent or broadcast by the other parties in the same round.

We consider t -bounded, static, active adversaries. That is, adversaries who may corrupt up to t parties, from the outset of a protocol, with the ability to read all incoming messages and internal values of that party and determine its output messages. We always assume a *rushing* adversary; i.e., in any round the malicious parties receive the messages sent by the honest parties before deciding their own.

2.2 Cyclotomic Rings, Norms, and Rounding

Consider the cyclotomic rings $R = \mathbb{Z}[x]/\phi$ and $R_q = \mathbb{Z}_q[x]/\phi$, where $\phi = (x^N + 1)$ for N an integer power of 2 and q a prime. Elements in both rings are polynomials of degree at most $N - 1$, with those in the latter ring having coefficients between $-(q - 1)/2$

and $(q - 1)/2$. We denote elements of $\mathbb{Z}$ and R by lower-case letters, vectors in R^k by bold lower-case letters, and matrices in $R^{(k \times \ell)}$ by bold upper-case letters. We use the standard notation $\|\cdot\|$ for norms of ring elements or vectors, specifying with a prefix which norm is meant (1, 2, or ∞). For a matrix $\mathbf{A} \in R^{k \times \ell}$, $\|\mathbf{A}\| \leq B$ denotes that *each entry* of the matrix has norm bounded by B .

We let $C_v = \{c \in R_q : \|c\|_\infty = 1, \|c\|_1 = v\} \subset R_q$ to be the set of polynomials with coefficients in $\{-1, 0, 1\}$ and exactly v non-zero coefficients, and define $\tilde{C}_v = \{c - c' : c, c' \in C_v, c \neq c'\}$. Let $\lfloor \cdot \rfloor : \mathbb{R} \rightarrow \mathbb{Z}$ denote the rounding-up operation. We define the lower κ bit dropping operation $\lfloor \cdot \rfloor_{q'} : \mathbb{Z}_q \rightarrow \mathbb{Z}_{q'}$ such that $\lfloor x \rfloor_{q'} := \lfloor x/2^\kappa \rfloor + q'/2^\kappa$ where $q' = \lfloor q/2^\kappa \rfloor$ for $\kappa \neq 0 \in \mathbb{N}$. Like other ring operations, the bit dropping operation extends into vectors naturally by applying everything coefficient-wise. We also define the map coefficient embedding by the map $\Phi : R_q \rightarrow \mathbb{Z}_q^N$ which maps elements of R_q to their coefficient vectors. This naturally extends to vectors and matrices of elements in R_q .

2.3 Lattice Assumptions

The security of our scheme relies on computational problems over structured lattices, namely Module Short Integer Solution (MSIS), Self-Target Module Integer Short Solution (SelfTargetMSIS), Module Learning with Errors (MLWE), and Hint Module Learning with Errors (H-MLWE) over some fixed polynomial ring R_q implicitly defining the parameters q and d . We present SelfTargetMSIS and H-MLWE here, and refer to Appendix B.3 for MSIS and MLWE.

Definition 1 (SelfTargetMSIS [36]). *Let k, ℓ be positive integers and $0 < \eta \ll q$. Let $H : R_q^k \times \{0, 1\}^* \rightarrow C_v$ be a cryptographically secure hash function modeled as a random oracle. Then, given $(\mathbf{A}, \mathbf{t}) \in R_q^{k \times \ell} \times R_q^k$, the Self-Target MSIS problem asks an adversary $\mathcal{A}$ to find $(\mathbf{z}, \mu) \in R_q^\ell \times \{0, 1\}^*$ such that $0 < \|\mathbf{z}\|_2 \leq \eta$ and $H([\mathbf{A} \mid \mathbf{t}] \cdot \mathbf{r}, \mu) = c$, where $\mathbf{r} = [\mathbf{z} \mid c]^\top$. $\mathcal{A}$ is said to have advantage ϵ_{STMSIS} in solving SelfTargetMSIS $_{k, \ell, \eta}$ if*

$$\Pr \left[\begin{array}{l} H([\mathbf{A} \mid \mathbf{t}] \cdot \mathbf{r}, \mu) = c \\ \wedge 0 < \|\mathbf{z}\|_2 \leq \eta \end{array} \mid \mathbf{A} \leftarrow R_q^{k \times \ell}; \left(\mathbf{r} = \begin{bmatrix} \mathbf{z} \\ c \end{bmatrix}, \mu \right) \leftarrow \mathcal{A}^H(\mathbf{A}, \mathbf{t}) \right] \geq \epsilon_{\text{STMSIS}}.$$

Definition 2 (H-MLWE [53]). *Let k, ℓ, Q be positive integers, χ_1 and χ_2 be probability distributions over R_q , and C be a subset of R_q . The Hint-MLWE problem H-MLWE $_{k, \ell, \chi_1, \chi_2, Q}$ then asks an adversary $\mathcal{A}$ to distinguish between the following two cases:*

- (1) $(\mathbf{A}, \mathbf{As}, (c_i, \mathbf{z}_i)_{i \in [Q]})$ for $\mathbf{A} \leftarrow R_q^{k \times (\ell+k)}$,
- (2) $(\mathbf{A}, \mathbf{b}, (c_i, \mathbf{z}_i)_{i \in [Q]})$ for $\mathbf{A} \leftarrow R_q^{k \times (\ell+k)}$, $\mathbf{b} \leftarrow R_q^k$,

where $\mathbf{s} \leftarrow \chi_1^{\ell+k}$, $c_i \leftarrow C$ for $i \in [Q]$, and $\mathbf{z}_i := c_i \cdot \mathbf{s} + \mathbf{y}_i$ where $\mathbf{y}_i \leftarrow \chi_2^{\ell+k}$ for $i \in [Q]$. We denote by $\epsilon_{\text{H-MLWE}}$ the advantage of $\mathcal{A}$ in solving H-MLWE $_{k, \ell, \chi_1, \chi_2, Q}$.

$\mathcal{A}$ has advantage $\epsilon_{\text{H-MLWE}}$ in solving H-MLWE $_{k, \ell, \chi_1, \chi_2, Q}$ if

$$\left| \Pr \left[\begin{array}{l} \mathbf{A} \leftarrow R_q^{k \times (\ell+k)}; \mathbf{s} \leftarrow \chi_1^{\ell+k}; c_i \leftarrow C; \\ b = 1 \mid \mathbf{y}_i \leftarrow \chi_2^{\ell+k}; \mathbf{z}_i := c_i \mathbf{s} + \mathbf{y}_i \text{ for } i \in [Q]; \\ b \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{As}, (c_i, \mathbf{z}_i)_{i \in [Q]}) \end{array} \right] \right. \\ \left. - \Pr \left[\begin{array}{l} \mathbf{A} \leftarrow R_q^{k \times (\ell+k)}; \mathbf{b} \leftarrow R_q^k; \\ b = 1 \mid \mathbf{s} \leftarrow \chi_1^{\ell+k}; c_i \leftarrow C; \mathbf{y}_i \leftarrow \chi_2^{\ell+k}; \\ \mathbf{z}_i := c_i \mathbf{s} + \mathbf{y}_i \text{ for } i \in [Q]; \\ b \leftarrow \mathcal{A}(\mathbf{A}, \mathbf{b}) \end{array} \right] \right| \geq \epsilon_{\text{MLWE}}.$$

2.4 The BDLOP Commitment Scheme

We recall the commitment scheme introduced in [9] which allows one to commit to vectors $\mathbf{m} \in R_q^{\ell_m}$ of ‘fully fledged’ elements in R_q . The scheme is parameterized by the system parameters sp_{Com} containing the natural numbers $k_{\text{MSIS}}, k_{\text{MLWE}}, \ell_m, q, v \in \mathbb{N}$ and real number $\sigma_{\text{Com}} \in \mathbb{R}^+$.

Challenge Space. Let v be such that $\binom{N}{v} \cdot 2^v > 2^\lambda$. We define the challenge space $C_v := \{c \in R_q \mid \|c\|_\infty \leq 1 \wedge \|c\|_1 \leq v\}$ and its difference set $\tilde{C}_v := \{c - c' \mid c, c' \in C_v \wedge c \neq c'\}$.

BDLOP.Setup(sp_{Com}) : On input parameters sp_{Com} , samples uniform $A_1, A_2 \in R_q^{k_{\text{MSIS}} \times (k_{\text{MSIS}} + k_{\text{MLWE}} + \ell_m)} \times R_q^{\ell_m \times (k_{\text{MSIS}} + k_{\text{MLWE}} + \ell_m)}$ outputs the public commitment key pk_{Com} defined as:

$$\text{pk}_{\text{Com}} := \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$$

BDLOP.Com($\text{pk}_{\text{Com}}, \mathbf{m}$) : On input a public commitment key pk_{Com} and a vector $\mathbf{m}$ in $R_q^{\ell_m}$, samples a vector $r_m \leftarrow \mathcal{D}_{\sigma_{\text{Com}}}^{(k_{\text{MSIS}} + k_{\text{MLWE}} + \ell_m)}$ and computes the commitment:

$$\text{com}_m := \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} r_m + \begin{bmatrix} 0 \\ \mathbf{m} \end{bmatrix} \in R_q^{k_{\text{MSIS}} + \ell_m}.$$

It then outputs com_m and the opening $d = (\mathbf{m}, r_m, 1)$.

BDLOP.Open($\text{pk}_{\text{Com}}, \text{com}, d$) : On input a public commitment key pk_{Com} , the commitment com and the opening $d = (\mathbf{m}, r_m, f)$ where $f \in \tilde{C}$. It verifies:

$$f \cdot \text{com} \stackrel{?}{=} \begin{bmatrix} A_1 \\ A_2 \end{bmatrix} r_m + \begin{bmatrix} 0 \\ f \cdot \mathbf{m} \end{bmatrix} \in R_q^{k_{\text{MSIS}} + \ell_m}.$$

and $\|r_m\|_2 \leq t_2 \cdot \sigma_{\text{Com}} \cdot \sqrt{N}$ (norms are calculated entry-wise). It outputs 1 if the relations hold and 0 otherwise.

Lemma 1 (Commitment Security [9]). *Define $k := k_{\text{MSIS}} + \ell_m$, $\ell := k_{\text{MSIS}} + k_{\text{MLWE}} + \ell_m$, and $\beta := 16 \cdot \sigma_{\text{Com}} \cdot \sqrt{v} \cdot N$. The BDLOP commitment scheme is hiding if $\text{MLWE}_{k, \ell, \mathcal{D}_{\sigma_{\text{Com}}}}$ is hard and it is binding if $\text{MSIS}_{k_{\text{MSIS}}, \ell, \beta}$ is hard.*

2.5 Threshold Signatures

A threshold signature scheme $\mathcal{S}$ (adapting [57, Section 4] and [28, Definition 5]) consists of the following algorithms:

- $\text{KGen}_{\mathcal{S}}$ is an interactive protocol run by n users that takes as input n and a threshold t . Each user either aborts or outputs a (common) public key $\text{pk}_{\mathcal{S}}$, (common) auxiliary data aux , and a secret key share $\text{sk}_{\mathcal{S}}^{(i)}$.
- $\text{Sign}_{\mathcal{S}}$ is an interactive protocol run by a set $\mathcal{U}$ of t users. Each party begins holding their secret key share, auxiliary data aux , and a message μ . Each user either aborts or outputs a signature Ψ .
- $\text{Vf}_{\mathcal{S}}$ takes as input the public key $\text{pk}_{\mathcal{S}}$, a message μ , and a signature Ψ , and outputs 1 iff the signature is valid.

Threshold Correctness We say that a threshold signature scheme $\mathcal{S}$ is correct if for all thresholds (t, n) and security parameter λ we have that

- for all $(\text{pk}_{\mathcal{S}}, \{\text{aux}_i, \text{sk}_{\mathcal{S}}^{(i)}\}_{i \in [n]}) \leftarrow \text{KGen}_{\mathcal{S}}(t, n, \lambda)$

- for all $\Psi \leftarrow \text{Sign}_{\mathcal{S}}(\mathcal{U}, \mu, \text{pk}_{\mathcal{S}}, \{\text{aux}_j, \text{sk}_{\mathcal{S}}^{(j)}\}_{j \in \mathcal{U}})$, where $\mathcal{U} \subset [n]$ and $|\mathcal{U}| \geq t$, we have that $1 \leftarrow \text{Vf}_{\mathcal{S}}(\text{pk}_{\mathcal{S}}, \mu, \Psi)$.

The security notion we consider is threshold unforgeability:

Threshold Unforgeability $\mathcal{S}$ is (T, ϵ) -threshold unforgeable if the probability of any adversary $\mathcal{A}$ running in time T succeeding in experiment $\text{Exp}_{\mathcal{S}}^{\text{ts-uf-cma}}(\mathcal{A})$ as depicted in Figure 1 is at most ϵ .

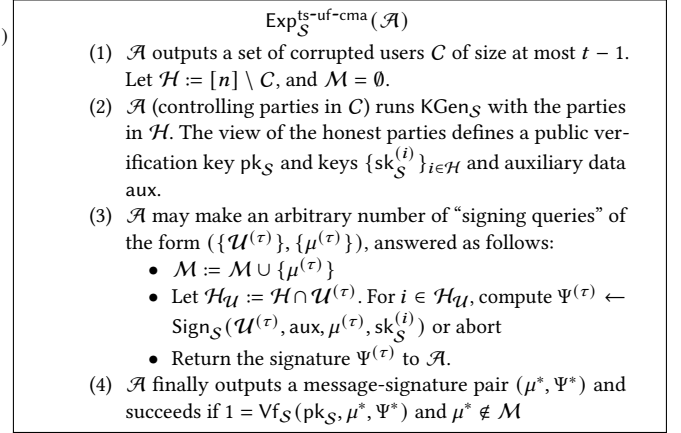


Figure 1: Experiment $\text{Exp}_{\mathcal{S}}^{\text{ts-uf-cma}}(\mathcal{A})$ parameterized by threshold t and total number of users n .

Moreover, we make the security against identifiable aborts explicit based on the definition of [49]:

Security with Identifiable Aborts $\mathcal{S}$ has (T, ϵ) -security with identifiable aborts if the probability of any adversary $\mathcal{A}$ running in time T succeeding in experiment $\text{Exp}_{\mathcal{S}}^{\text{ID-AB}}(\mathcal{A})$ as depicted in Figure 2 is at most ϵ .

We remark that this definition of security with identifiable aborts considers only the signing protocol $\text{Sign}_{\mathcal{S}}$, and not the distributed key generation protocol $\text{KGen}_{\mathcal{S}}$.⁷

2.6 Coding Theory

Here we recall the basic notions and results from coding theory needed in our verifiable secret sharing.

Definition 3 (Linear Error Correcting Code). *We define an $[n, k, d]$ code C is a linear error correcting code over $\mathbb{Z}_q$ of length n , dimension k and minimum distance d . Its dual code $C^\perp$ is the vector space consisting of all vectors $c^\perp \in \mathbb{Z}_q^n$ such that $\langle c, c^\perp \rangle = 0$ for all $c \in C$.*

We will focus on Reed-Solomon codes as following.

Definition 4 (Reed-Solomon (RS) Code). *An code C is a Reed-Solomon (RS) code if it is of the following form*

$$C := \{(p(1), p(2), \dots, p(n)) : p(x) \in \mathbb{Z}_q[x], \deg(p(x)) \leq k - 1\}$$

where $p(x)$ is any polynomial in $\mathbb{Z}_q[x]$ of degree at least $k - 1$.

⁷This will also be the case when we introduce a similar notion for the homomorphic encryption scheme.

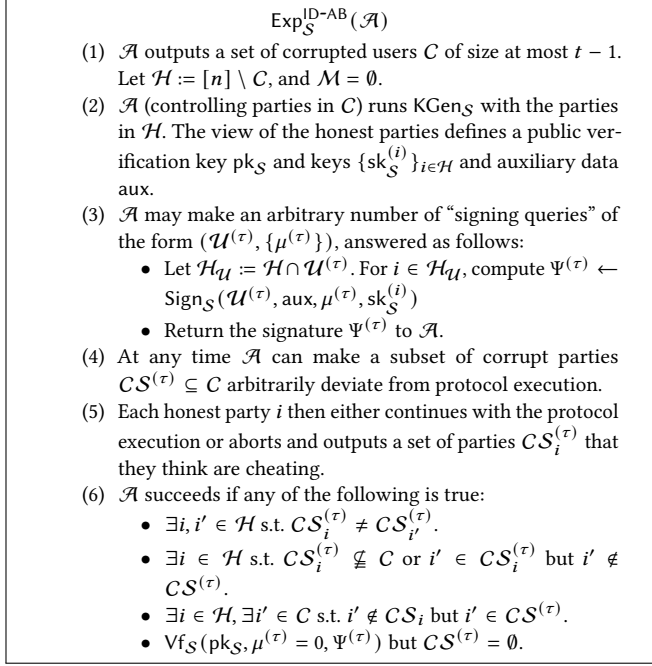


Figure 2: Experiment $\text{Exp}_S^{\text{ID-AB}}(\mathcal{A})$ parameterized by threshold t and total number of users n .

We naturally extend this notation to accommodate polynomials $\mathbf{P} \in R_q^{m_1 \times m_2}[x]$. That is, polynomials whose coefficients are matrices of elements in R_q and whose domain is $\mathbb{N}$ and range is $R_q^{m_1 \times m_2}$. Then we can define an $[n, k, s]$ RS code C as

$$C := \{(\mathbf{P}(1), \dots, \mathbf{P}(n)) : \mathbf{P}(x) \in R_q^{m_1 \times m_2}[x], \deg(\mathbf{P}(x)) \leq k - 1\} \quad (1)$$

Note, Equation (1) can be viewed as capturing $N \cdot m_1 \cdot m_2$ codes with a more compact notation. We will use the following (adapted) lemma from [23]. Note that the dual code remains defined over $\mathbb{Z}_q^n$.

Lemma 2 ([23] (adapted)). *If $S_1, \dots, S_n \in R_q^{m_1 \times m_2} \setminus C$, and $c^\perp$ is chosen randomly in $C^\perp$ then the following holds:*

$$\Pr \left[\sum_{i=1}^n c^\perp S_i = \mathbf{0} \right] = \frac{N \cdot m_1 \cdot m_2}{q}.$$

3 Verifiable Secret Sharing

Verifiable secret sharing (VSS), introduced by Chor et al. [26], can be viewed as an interactive protocol for secret sharing that is secure against malicious behavior by all parties (including the dealer) during both the sharing and reconstruction phases. One of the most common designs combines the distribution (by a dealer) of committed (or encrypted) shares along with zero-knowledge proofs that the commitments contain correct shares. Our lattice-based design follows this framework and is thus closer in design to the constructions of Feldman [39] and Pedersen [64] in the discrete logarithm setting.

Gentry et al. [45] proposed a publicly-verifiable secret sharing (PVSS) scheme from lattice-based encryption and Bulletproofs [22].

Here, publicly verifiable means that anyone can verify that shares have been generated and distributed correctly. While lattice-based encryption provides post-quantum secrecy with respect to the ciphertexts, Bulletproofs are built from classical assumptions, so the proof aspect opens the door to a quantum adversary. Recently, Espitau et al. [38] proposed a fully lattice-based VSS. Their work introduced a new ‘flood-and-submerge’ technique to build a robust VSS. Unfortunately, security is only guaranteed for an honest majority ($n \geq 2t + 1$) and is thus unsuitable in our setting.

We propose an efficient (fully) lattice-based VSS, secure for arbitrary thresholds ($t < n$). Our design takes inspiration from the above schemes, whereby a dealer commits to the set of shares and broadcasts these with a non-interactive proof of opening. They then distribute the shares (openings) among the shareholders via secure channels. Shareholders can identify a faulty dealing by checking that their share is a valid opening for their commitment.

In order to make the non-interactive proof of opening efficient, we employ an observation of McEliece and Sarwate [61]: Shamir secret sharing [71] is equivalent to encoding the secret using a Reed-Solomon error correcting code. In particular, the vector of secret shares forms a codeword of a Reed-Solomon code. As observed by Cascudo and David in [23], to prove a Shamir secret sharing, it suffices to prove that the inner product of this codeword with a randomly selected word from the dual code is zero. In [23], the discrete log setting lends itself naturally to allowing anyone to compute the inner product of a random codeword of their choice with the set of committed shares. This naturally preserves the secrecy of shares as the check is performed in the exponent. In the lattice setting, however, there is no such structure. The solution we propose, is to demonstrate the inner product check in zero-knowledge where the random codeword is provided as part of the random challenge of the proof protocol. Anyone verifying such a proof can be confident with overwhelming likelihood that the committed shares are correct.

3.1 VSS Model and Definitions

We assume the existence of secure channels between the dealer and each shareholder. We also assume, for generality, the dealer’s input lies in $R_q^{m_1 \times m_2}$ where $\mathbb{Z}_q$ contains $\{1, \dots, n\}$.

We present a 1-round (non-interactive) VSS design with a slight relaxation that the output of the honest parties can be a pre-defined abort symbol. This variant is called *weak* verifiable secret sharing (WSS) [52, 68, 69]. Looking ahead, our threshold signature scheme uses a 1-round DKG for which robustness cannot be guaranteed; that is, a well-formed output cannot be guaranteed for honest parties. Thus, WSS is sufficient in our setting.

We define a weak verifiable secret sharing (WSS) scheme.

Definition 5 (Weak verifiable secret sharing). *A two-phase (sharing and reconstruction) protocol involving n parties $\mathcal{P} := \{\mathcal{P}_1, \dots, \mathcal{P}_n\}$ in which a dealer $D \in \mathcal{P}$ holds an initial input $S \in R_q^{m_1 \times m_2}$, is a WSS tolerating $t - 1$ malicious parties if the following properties hold with overwhelming probability in the presence of any (PPT) adversary controlling at most $t - 1$ parties:*

Privacy *If the dealer is honest at the end of the sharing phase, then at the end of this phase the joint view of the malicious parties is independent of the dealer’s input S .*

Correctness If the dealer is honest, with input S , then at the end of the sharing phase each honest party $\mathcal{P}_i$ outputs a value $S_i \in R_q^{m_1 \times m_2}$ with $S_i \neq \perp$. If $\geq t$ honest parties subsequently run the reconstruction phase, all output S .

Weak commitment For a malicious dealer, at the end of the sharing phase the joint view of the honest parties defines a value $S^* \in R_q^{m_1 \times m_2} \cup \perp$ (which can be computed in polynomial time from this view) such that if any set of $\geq t$ parties subsequently runs the reconstruction phase, either every honest party outputs S^* or every honest party outputs $\perp$. Moreover, assuming each honest party outputs some $S_i \neq \perp$ at the end of the sharing phase, there exists a polynomial $P(x) \in R_q^{m_1 \times m_2}[X]$ of degree at most t such that:

- (1) $S_i = P(i)$ for every honest party $\mathcal{P}_i$.
- (2) if $S^* \neq \perp$, then $S^* = P(0)$.

3.2 Overview of our VSS Construction

Our (P)VSS construction relies on three main building blocks.

Shamir Secret Sharing. Shamir Secret Sharing [71] is used by the dealer to generate shares of its input S .

BDLOP Commitment. The dealer broadcasts commitments to the shares using the lattice-based BDLOP [9] commitment scheme described in Section 2.4.

Non-interactive Zero Knowledge. We employ a novel combination of zero-knowledge proofs to enforce the verifiability property of the secret sharing. The dealer begins by committing to the secret S , as well as all shares S_i using the BDLOP commitment scheme. The dealer then proves that

- π_1 S has small norm using the LNP framework in [59]
- π_2 The set of shares $\{S_i\}$ forms a codeword of a corresponding Reed-Solomon code. This is done using the Π_{LIN} protocol from [9]. Explicitly, we prove that for a random element $c^\perp = (c_1, \dots, c_n) \in \mathbb{Z}_q^n$ of the dual code,

$$\sum_{i=1}^n c_i^\perp \cdot S_i = 0.$$

- π_3 A t -sized subset of the shares reconstructs correctly to the secret. Again, this uses Π_{LIN} . Explicitly, given a set $\mathcal{S} \subset [n]$ of size t , we prove that

$$S = \sum_{i \in \mathcal{S}} \lambda_i^{\mathcal{S}} S_i,$$

where the $\lambda_i^{\mathcal{S}}$ are the Lagrange coefficients.

Crucially, we note that proofs π_2 and π_3 ensure that the committed objects form a correct sharing. Note that proof π_2 argues that the $\{S_i\}$ are a correct sharing of *something* while proof π_3 argues that some t -sized subset of the shares correctly reconstructs S . Crucially, the two together prove that *any* t -sized subset of the shares will reconstruct the secret.

We note that our design could be made publicly verifiable if needed. In particular, the dealer could employ a PKE scheme such as CRYSTALS-Kyber [70] to encrypt shares to shareholders. These ciphertexts could be broadcast along with proof that their the underlying plaintexts satisfy a correct sharing. This verifiable encryption

Protocol Π_{VSS}

Let C be an error-correcting code corresponding to the t -out-of- n -threshold Shamir Secret Sharing scheme and let $C^\perp$ be its dual code. Let H be a random oracle mapping $\{0, 1\}^*$ to $C^\perp$. Protocol Π_{VSS} is run between n parties $\mathcal{P}_1, \dots, \mathcal{P}_n$, a dealer D and an external verifier V . Let $\mathcal{D}_{\mathbf{B}}$ be a distribution over $R_q^{m_1 \times m_2}$.

Setup: The $\text{BDLOP.Setup}(\text{sp}_{\text{Com}})$ commitment setup algorithm is run to generate the public commitment key pk_{Com} of the BDLOP commitment scheme, in practice generated from a seed.

Distribution: The dealer D first samples the secret $S \leftarrow \mathcal{D}_{\mathbf{B}}$ and does the following

- Sample $P \leftarrow R_q^{m_1 \times m_2}[X]$ of degree $< t$, with $P(0) = S$.
- For $i \in [n]$, compute the i 'th share as $\llbracket S \rrbracket_i := P(i)$.
- For $i \in [n]$, $(\text{com}_{\llbracket S \rrbracket_i}, d) \leftarrow \text{BDLOP.Com}(\text{pk}_{\text{Com}}, \llbracket S \rrbracket_i)^a$.
- Compute $c^\perp := H(\{(\text{com}_{\llbracket S \rrbracket_i})\}_{i \in [n]})$.
- Generate $\pi_{\text{VSS}} := (\pi_1, \pi_2, \pi_3)$, where π_2 is computed w.r.t. $c^\perp$ and π_3 is generated w.r.t. $\mathcal{S} := \{1, \dots, t\}$ (w.l.o.g.).
- Broadcast π_{VSS} and send $(\llbracket S \rrbracket_i, r_{\llbracket S \rrbracket_i})$ to party i over a secure channel.

Upon (secure) receipt of $d := (\llbracket S \rrbracket_i, r_{\llbracket S \rrbracket_i}, 1)$, party i checks that $c^\perp := H(\{(\text{com}_{\llbracket S \rrbracket_i})\}_{i \in [n]})$ and then verifies the proof π_{VSS} (each constituent proof) and that d is a correct opening of $\text{com}_{\llbracket S \rrbracket_i}$, outputting abort ($\perp$) if either fails.

Reconstruction: Suppose a subset $\mathcal{S}$ of size t wish to reconstruct the secret S . If verification of π_{VSS} failed in the sharing phase, $\mathcal{P}_i$ will output $\perp$ here also. Assuming no party outputs $\perp$, each party $\mathcal{P}_i \in \mathcal{S}$ broadcasts $(\llbracket S \rrbracket_i, r_{\llbracket S \rrbracket_i})$. Upon receiving all shares and openings from parties $\mathcal{P}_j$ ($j \neq i$), $\mathcal{P}_i$ checks that $\text{Open}(\text{com}_{\llbracket S \rrbracket_j}, d := (\llbracket S \rrbracket_j, r_{\llbracket S \rrbracket_j}, 1)) = 1$ for all $j \in \mathcal{S} \setminus \{i\}$, aborting otherwise. The secret S is then reconstructed as

$$S = \sum_{i \in \mathcal{S}} \lambda_i^{\mathcal{S}} \llbracket S \rrbracket_i.$$

^a $\text{com}_{\llbracket S \rrbracket_i}$ is the horizontal concatenation of commitments to the columns of $\llbracket S \rrbracket_i$

Figure 3: Verifiable secret sharing protocol Π_{VSS}

design was used in [45]. For our applications, a commit-and-prove design is sufficient and makes for a simpler presentation.

We do however allow for an additional ‘complaint’ round in which parties may broadcast an abort symbol should they find that their received share is not a valid opening of the dealer’s corresponding commitment.

We present the VSS scheme Π_{VSS} in Figure 3.

3.3 Security of VSS

We now show the theorem our VSS scheme provides and provide proofs for this claim in three lemmas in Appendix C.1.

Theorem 1 (VSS Security). *The WSS scheme Π_{VSS} offers correctness, privacy, and weak commitment.*

4 Threshold Homomorphic Encryption

We present the underlying threshold linearly homomorphic encryption scheme. This follows a similar structure to the one used in [48] but is now defined over module lattices with the aim of enabling more efficient parameters later on. Since we employ a distributed key generation and (non-interactive) threshold decryption, our encryption scheme is defined by the tuple of algorithms $(\text{KGen}_{\mathcal{E}}, \text{Enc}, \text{Eval}, \text{TDec}, \text{Comb})$. We begin by describing the distributed key generation algorithm $\text{KGen}_{\mathcal{E}}$ before defining the remaining algorithms, with the omission of Eval which we will define according to its specific function in our threshold signature scheme.

4.1 Distributed Key Generation

We define a t -out-of- n distributed key generation protocol based on a module lattice encryption scheme based on CRYSTALS-Kyber [70] and BGV [20], closely following the construction by Gur et al. [48]. Due to this similarity, we present the key generation algorithm in its full, actively secure form.

Our threshold homomorphic encryption scheme is defined with respect to the public parameters $\text{sp} = (k, \ell, \hat{k}, \hat{\ell}, m, q, \hat{q}, N, \sigma_{\text{LWE}})$, where $k, \ell, \hat{k}, \hat{\ell}, m, q, \hat{q}, N \in \mathbb{N}$, $\sigma_{\text{LWE}} \in \mathbb{R}^+$, and σ_{LWE} defines the discrete Gaussian distributions $\mathcal{D}_{\mathbf{B}}$ and $\mathcal{D}_{\text{ctx}}$ (used in the secret sharing). It is helpful to think of (k, ℓ, q) as being smaller and defining an ‘inner layer’ of signing keys (and plaintext space) and $(\hat{k}, \hat{\ell}, m, \hat{q})$ as being larger and defining an ‘outer layer’ for a ciphertext space.

We assume all parties agree on a common $\mathbf{A}_{\mathcal{E}} \in R_q^{k \times \hat{\ell}}$ which could be the output of a random oracle on some session ID or nonce. Let H be a hash function acting as a random oracle where $H : \mathbb{N} \times R_q^{k \times m} \rightarrow \{0, 1\}^K$. We describe the distributed key generation algorithm from the point of view of a party $\mathcal{P}_i$;

1. $\mathcal{P}_i$ samples short $\mathbf{S}_i \leftarrow \mathcal{D}_{\mathbf{B}}^{\hat{\ell} \times m}$ and $\mathbf{E}_i \leftarrow \mathcal{D}_{\mathbf{B}}^{\hat{\ell} \times m}$, computes $\mathbf{B}_i := \mathbf{A}_{\mathcal{E}} \mathbf{S}_i + q \mathbf{E}_i$ and $h_i \leftarrow H(i, \mathbf{B}_i)$, and broadcasts h_i .
2. On receipt of $\{h_j\}$ for $j \neq i$, $\mathcal{P}_i$ generates t -out-of- n Shamir secret shares $\{S_{i,j}\}_{j \in [n]} := \text{Share}_{t,n}(\mathbf{S}_i)$ and $\{E_{i,j}\}_{j \in [n]} := \text{Share}_{t,n}(\mathbf{E}_i)$, and sets $\mathbf{B}_{i,j} := \mathbf{A}_{\mathcal{E}} S_{i,j} + q E_{i,j}$, for $j \in [n]$.
3. $\mathcal{P}_i$ now creates a zero-knowledge proof $\pi_{\text{KGen}_{\mathcal{E}}}^{(i)}$ of witness

$$W := \left(\mathbf{S}_i, \mathbf{E}_i, \{S_{i,j}\}_{j \in [n]}, \{E_{i,j}\}_{j \in [n]} \right),$$

such that (1) $\mathbf{B}_i$ and $\{\mathbf{B}_{i,j}\}_{j \in [n]}$ are constructed correctly w.r.t. W , (2) $\{S_{i,j}\}_{j \in [n]}$ (resp. $\{E_{i,j}\}_{j \in [n]}$) are a correct t -out-of- n secret sharing of $\mathbf{S}_i$ (resp. $\mathbf{E}_i$), and (3) $\|\mathbf{S}_i\|_2, \|\mathbf{E}_i\|_2 < t_2 \cdot \mathbf{B} \cdot \sqrt{N}$. (1) is performed using the Π_{LIN} protocol from [9] whilst (2) and (3) are performed by the proof π_{VSS} as described in Section 3.2. We denote by $\pi_{\text{KGen}_{\mathcal{E}}}^{(i)}$ the collection of these proofs. $\mathcal{P}_i$ samples commitment randomness $\rho_{i,j} \leftarrow \chi_{\rho}$ and commits to $S_{i,j}$ as $\text{com}_{i,j} := \text{Com}(S_{i,j}, \rho_{i,j})$.⁸ $\mathcal{P}_i$

⁸In practice, $\pi_{\text{KGen}_{\mathcal{E}}}^{(i)}$ contains such a commitment but we make it explicit in the protocol description since it will be used in our signature scheme. We also note that commitment to a matrix here can be done by committing to the vector of polynomials formed by the vertical concatenation of its columns.

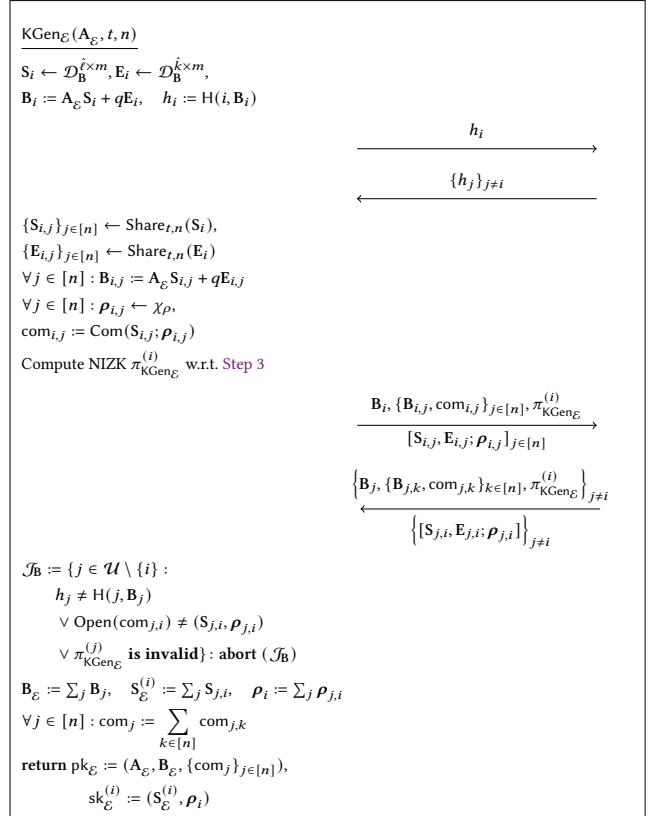


Figure 4: Actively secure key-generation protocol, from the point of view of $\mathcal{P}_i$. The elements in square brackets with subscript j are sent to $\mathcal{P}_j$ over a private channel.

sends $\{S_{i,j}, E_{i,j}, \rho_{i,j}\}_{j \in [n]}$ to $\mathcal{P}_j$ (via a secure channel) and broadcasts $(\mathbf{B}_i, \{\mathbf{B}_{i,j}, \text{com}_{i,j}\}_{j \in [n]}, \pi_{\text{KGen}_{\mathcal{E}}}^{(i)})$.

4. $\mathcal{P}_i$ defines $\mathcal{J}_{\mathcal{B}} := \{j \in \mathcal{U} \setminus \{i\} : h_j \neq H(j, \mathbf{B}_j) \vee \text{Open}(\text{com}_{j,i}) \neq (S_{j,i}, \rho_{j,i}) \vee \pi_{\text{KGen}_{\mathcal{E}}}^{(j)} \text{ is invalid}\}$. If $\mathcal{J}_{\mathcal{B}} \neq \emptyset$, $\mathcal{P}_i$ aborts with output $\mathcal{J}_{\mathcal{B}}$ (and subsequently all parties abort)⁹. Assuming no party aborts, $\mathcal{P}_i$ computes $\mathbf{B}_{\mathcal{E}} := \sum_j \mathbf{B}_j$, $\mathbf{S}_{\mathcal{E}}^{(i)} := \sum_j S_{j,i}$, and $\rho_i := \sum_j \rho_{j,i}$. $\mathcal{P}_i$ then computes the public commitments $\text{com}_j := \sum_{k \in [n]} \text{com}_{j,k}$. The public key is $\text{pk}_{\mathcal{E}} := (\mathbf{A}_{\mathcal{E}}, \mathbf{B}_{\mathcal{E}}, \{\text{com}_j\}_{j \in [n]})$ and secret key $\text{sk}_{\mathcal{E}}^{(i)} := (\mathbf{S}_{\mathcal{E}}^{(i)}, \rho_i)$.

4.2 Threshold Decryption

We now describe how threshold decryption is performed. Inheriting the parameter set from the previous section, we additionally define the Gaussian parameter σ_{TDec} . Figure 5 displays the threshold decryption and combination algorithms TDec and Comb respectively. We also include the BGV-style encryption algorithm Enc , encrypting a vector of length m to make ciphertext structure explicit.

⁹We note that while this provides some level of identifiable aborts, it does not completely satisfy the ID-A property. However, with the use of verifiable encryption we believe this to be possible, though that is left for future work.

Encryption $\text{Enc}(\text{pk}_E, \mathbf{m})$. Given public key $\text{pk}_E = (\mathbf{A}_E, \mathbf{B}_E) \in R_q^{\hat{k} \times \hat{\ell}} \times R_q^{\hat{k} \times m}$ and message $\mathbf{m} \in R_q^m$:

- (1) Sample randomness $\mathbf{r} \leftarrow \mathcal{D}_{\text{ctx}}^{\hat{k}}$ and noise terms $\mathbf{e}_1 \leftarrow \mathcal{D}_{\text{ctx}}^{\hat{\ell}}$ and $\mathbf{e}_2 \leftarrow \mathcal{D}_{\text{ctx}}^m$.
- (2) Compute $\mathbf{u} := \mathbf{A}_E^\top \mathbf{r} + q \cdot \mathbf{e}_1$ and $\mathbf{v} := \mathbf{B}_E^\top \mathbf{r} + q \cdot \mathbf{e}_2 + \mathbf{m}$.
- (3) Return ciphertext $\text{ctx} = (\mathbf{u}, \mathbf{v})$.

Threshold Decryption $\text{TDec}(\text{sk}_E^{(i)}, \text{ctx}, \mathcal{U})$. Given decryption key share $\text{sk}_E^{(i)} = (\mathbf{S}_E^{(i)}, \rho_i)$ and ciphertext $\text{ctx} = (\mathbf{u}, \mathbf{v})$:

- (1) Sample noise drowning term $E_i \leftarrow \mathcal{D}_{\sigma_{\text{TDec}}}^m$.
- (2) Compute Lagrange coefficient λ_i for party $\mathcal{P}_i$ w.r.t. $\mathcal{U}$.
- (3) Compute $\text{ds}_i := \lambda_i \mathbf{S}_E^{(i)\top} \mathbf{u} + q \cdot E_i$.
- (4) Return decryption share ds_i .

Combine $\text{Comb}(\{\text{ds}_i\}_{i \in \mathcal{U}}, \text{ctx})$. Given a set of decryption shares and a ciphertext:

- (1) Compute $\mathbf{m} := (\mathbf{v} - \sum_{i \in \mathcal{U}} \text{ds}_i) \bmod \hat{q} \bmod q$.
- (2) Return plaintext message $\mathbf{m}$.

Figure 5: The threshold decryption scheme MLWE-Encrypt.

4.3 Threshold Security

For completeness we first show correctness and then prove indistinguishability for the module variant of the encryption scheme.

Theorem 2. Let $\hat{q}/2 \geq B_{\text{Dec}} + t_{\infty} \cdot t \cdot \sigma_{\text{TDec}}$ and $\sigma_{\text{TDec}} \geq \sqrt{\frac{Q \cdot B_2^2}{q^2}}$ where B_2 and B_{Dec} are the ℓ_2 and ℓ_{∞} norm bounds for a correctly generated ciphertext ctx . Then the linearly homomorphic threshold decryption described in Section 4 correctly decrypts ctx .

Theorem 3. Let $a \in (1, +\infty)$ and $\sigma_{\text{TDec}} \geq \sqrt{\frac{Q \cdot B_2^2}{q^2}}$ where B_2 is the ℓ_2 norm bound of the noise in a correctly generated ciphertext ctx . Let $\text{MLWE}_{N,q,\sigma_{\text{LWE}}}$ be ϵ_{MLWE} hard, and module variant of BGV be $\epsilon_{\text{IND-CPA}}$ secure. Then the linearly homomorphic threshold encryption scheme $\mathcal{E}$ described in Section 4 is indistinguishable. More formally, for any PPT adversary $\mathcal{A}$ corrupting up to $t-1$ parties the advantage in the indistinguishability game can be bounded as:

$$\text{Adv}_{\mathcal{E}}^{\text{IND}}(\mathcal{A}) \leq \left(2^{\frac{a}{a-1}} \cdot (|\mathcal{H}| + |\mathcal{H}_{\mathcal{U}}| - 1) \cdot \epsilon_{\text{MLWE}} + \epsilon_{\text{IND-CPA}} \right)^{\frac{a-1}{2a-1}}.$$

We show the proof in Appendix C.2.

The following theorem asserts that the depicted protocol has security with identifiable aborts. This will be helpful later when we show our threshold signature scheme has security with identifiable aborts. Looking ahead for the signature scheme we assume each ds_i also comes with a proof $\pi_{\text{ds},i}$ that proves correct partial decryption.

Theorem 4. Let Π_x for language $\mathcal{R}_x$ be NIZKAoKs. Let Com be a binding commitment scheme and H be a hash function modeled as a random oracle. Then the actively secure threshold homomorphic encryption scheme $\mathcal{E}$ has security with identifiable aborts in the synchronous model with access to a broadcast channel.

The proof is given in Appendix C.3.

5 Threshold Signature Protocol

5.1 Underlying Rejection-Free Signatures

For the underlying signature we use a variant of the base signature of Threshold Raccoon [30]:

KGen Samples a uniform $\mathbf{A}_S \in R_q^{k \times \ell}$, a Gaussian secret key $\mathbf{s} \leftarrow \mathcal{D}_y^{\ell}$ and an error term $\mathbf{e} \leftarrow \mathcal{D}_y^k$. Finally, computes $\mathbf{y} := \mathbf{A}_S \mathbf{s} + \mathbf{e}$ and rounded version $\mathbf{y}' := \lfloor \mathbf{y} \rfloor_{q_y}$ where $q_y = \lfloor q/2^{k_y} \rfloor$, and outputs $\text{sk}_S := \mathbf{s}$ and $\text{pk}_S := (\mathbf{A}_S, \mathbf{y}')$.

Sign Takes as input $(\text{sk}_S, \text{pk}_S, \mu)$ and samples Gaussian masks $\mathbf{r} \leftarrow \mathcal{D}_w^{\ell}$, $\mathbf{e}' \leftarrow \mathcal{D}_w^k$. It then computes $\mathbf{w} := \mathbf{A}_S \mathbf{r} + \mathbf{e}'$ alongside a rounded version $\mathbf{w}' := \lfloor \mathbf{w} \rfloor_{q_w}$ where $q_w = \lfloor q/2^{k_w} \rfloor$, and derive the challenge $c := \text{H}(\text{pk}_S, \mathbf{w}', \mu) \in C_v$. Finally, it outputs the signature $\Psi := (c, \mathbf{z}, \mathbf{h})$ where $\mathbf{z} := \mathbf{c} \mathbf{s} + \mathbf{r}$, $\mathbf{h} := \mathbf{w}' - \mathbf{t}$, and $\mathbf{t} := \lfloor \mathbf{A}_S \cdot \mathbf{z} - 2^{k_y} \cdot c \cdot \mathbf{y}' \rfloor_{q_w}$.

Vf Takes as input $(\text{pk}_S, \Psi := (c, \mathbf{z}, \mathbf{h}), \mu)$ and checks that $0 < \|\mathbf{z}, 2^{k_w} \cdot \mathbf{h}\|_2 \leq B$ for some bound B and that $c = \text{H}(\text{pk}_S, \lfloor \mathbf{A}_S \cdot \mathbf{z} - 2^{k_y} \cdot c \cdot \mathbf{y}' \rfloor_{q_w} + \mathbf{h}, \mu)$. It outputs 1 if both hold and otherwise outputs 0.

It is easy to see that the signature above is the immediate realization of a digital signature using H-MLWE based zero-knowledge proofs [53] and the Fiat-Shamir transform [40] utilizing generic rounding based [7] approaches. Unforgeability of the signature scheme follows from a similar analysis as for the base signature of Threshold Raccoon [30]. For completeness, we recite the theorem:

Theorem 5 ([30, Theorem 5.1]). Let $q, N, k, \ell, \kappa_y, \kappa_w$ be integers, Q be number of signature queries the adversary can perform, and $B, B_{\text{MSIS}}, B_{\text{ST-MSIS}}, B_{\text{H-MLWE}}$ be bounds where $\sigma \geq \sqrt{\ell + k} \cdot \omega(\sqrt{\log N})$, $B = e^{1/4} \cdot (v \sigma_y + \sigma_w) \sqrt{N(k + \ell)} + (v \cdot 2^{k_y} + 2^{k_w+1}) \cdot \sqrt{Nk}$, $B_{\text{H-MLWE}} = Q \cdot v \cdot \left(1 + \frac{N(\ell+1+2 \log(N))}{\sqrt{Q}} \right)$, $\frac{1}{\sigma^2} = 2 \cdot \left(\frac{1}{n \cdot \sigma_y^2} + \frac{B_{\text{H-MLWE}}}{t \cdot \sigma_w^2} \right)$, $\sigma_y = 2\sqrt{\ell + k} \cdot \log N$, $\sigma_w = 2\sqrt{B_{\text{H-MLWE}} \cdot (\ell + k) \cdot \log N}$, $q > B_{\text{MSIS}} \sqrt{Nk} \cdot \omega(\log(Nk))$, $B_{\text{MSIS}} = 2B_{\text{ST-MSIS}}$, and $B_{\text{ST-MSIS}} = B + \sqrt{v} + (v \cdot 2^{k_y} + 2^{k_w+1}) \cdot \sqrt{Nk}$. Then the signature scheme is correct and unforgeable under the $\text{SelfTargetMSIS}_{k,\ell+1,v,B_{\text{ST-MSIS}}}$ and $\text{H-MLWE}_{k,\ell,\sigma_y,\sigma_w,Q}$ assumptions. More formally, for a PPT adversary $\mathcal{A}$ against the unforgeability game, Q_H and Q_{Sign} queries to random H and signing oracles respectively, there exists adversaries $\mathcal{B}$ and $\mathcal{B}'$ against $\text{H-MLWE}_{k,\ell,\sigma_y,\sigma_w,Q}$ and $\text{SelfTargetMSIS}_{k,\ell+1,v,B_{\text{ST-MSIS}}}$ such that the advantage of $\mathcal{A}$ can be bounded as:

$$\text{Adv}_{\mathcal{A}}^{\text{uf-cma}}(\mathcal{A}) \leq \text{Adv}_{\mathcal{B}}^{\text{H-MLWE}} + \text{Adv}_{\mathcal{B}'}^{\text{ST-MSIS}}(\mathcal{B}') + \frac{(Q_H+1) \cdot Q_{\text{Sign}}}{2^{N-1}}.$$

5.2 Actively Secure Threshold Signatures

We now present the actively secure, threshold signing protocol, constructed by bootstrapping the passively-secure protocol described in Section 1.2. Let $\text{H}_0, \text{H}_1, \text{H}_2$ be hash functions acting as random oracles where $\text{H}_0, \text{H}_1 : \mathbb{N} \times R_q^k \rightarrow \{0, 1\}^{\kappa}$ and H_2 is the same as H in Section 5.1. The key generation and signing protocols are depicted in Figure 6 and Figure 7, respectively.

We modify the design of [48], removing the need for trapdoor commitments at the cost of an extra round of communication and adding rounding to the public key and the signature. The motivation for this is to reduce the overall size of resulting signatures.

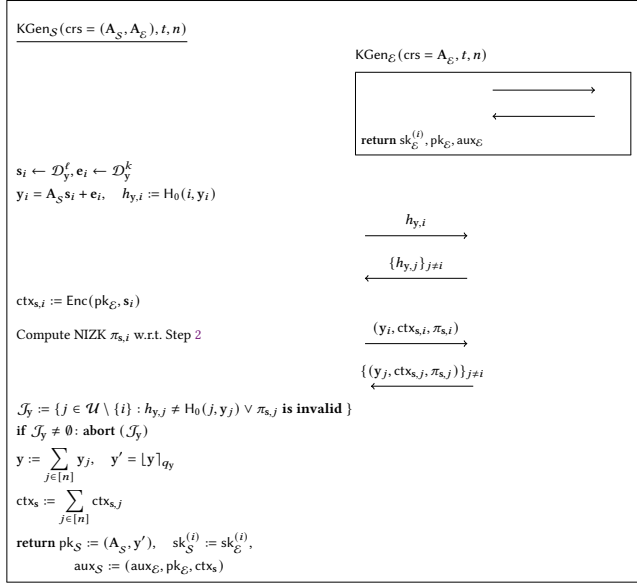


Figure 6: Actively secure key generation protocol with bit dropping for signer $\mathcal{P}_i$.

KGen_S works as follows:

- (1) $\mathcal{P}_i$ starts by invoking the distributed key generation KGen_E of the underlying encryption scheme $\mathcal{E}$ with inputs t and n as in the passive case and obtains the public encryption key pk_E , its threshold decryption key share $sk_E^{(i)}$ and any auxiliary information aux_E associated with $\mathcal{E}$.
- (2) $\mathcal{P}_i$ then samples short signing key $s_i \leftarrow D_y^\ell$ alongside an error $e_i \leftarrow D_y^k$ and sets $y_i := A_S s_i + e_i$, hash $h_{y,i} := H_0(i, y_i)$ as a commitment to y_i , and broadcasts $h_{y,i}$. Upon receiving $h_{y,j}$ for all $j \neq i$, $\mathcal{P}_i$ encrypts s_i as $ctx_{s,i} := \text{Enc}(pk_E, s_i)$ and computes a NIZK proof $\pi_{s,i}$ to prove that the correct s_i was used for y_i and $ctx_{s,i}$ (see Appendix D for the formal relations), then broadcasts $(y_i, ctx_{s,i}, \pi_{s,i})$.
- (3) Finally, upon receiving $ctx_{s,j}, \pi_{s,j}$ and y_j from each $j \neq i$, $\mathcal{P}_i$ defines $\mathcal{J}_y := \{j \in \mathcal{U} \setminus \{i\} : h_{y,j} \neq H_0(j, y_j) \vee \pi_{s,j} \text{ invalid w.r.t. } ctx_{s,j} \text{ and } y_j\}$. If $\mathcal{J}_y \neq \emptyset$, $\mathcal{P}_i$ aborts with output $\mathcal{J}_y$. Otherwise, it defines the public key $pk_S := (A_S, y')$, secret key $sk_S^{(i)} := sk_E^{(i)}$, and auxiliary information $aux_S = (aux_E, pk_E, ctx_s)$ where $y' := \lfloor \sum_{j \in \mathcal{U}} y_j \rfloor_{q_y}$ for $q_y = \lfloor q/2^{\kappa_y} \rfloor$, $ctx_s := \sum_{j \in \mathcal{U}} ctx_{s,j}$.

Sign_S works as follows:

- (1) Let $\mathcal{P}_i$ be one out of t parties in the set $\mathcal{U}$. Upon receiving the message μ to be signed, and $\mathcal{P}_i$ samples per signatures randomness $r_i \leftarrow D_w^\ell$ and $e'_i \leftarrow D_w^k$. It computes $w_i := A_S r_i + e'_i$ and $\mathcal{P}_i$ and $h_{w,i} := H_1(i, w_i)$, and broadcasts $h_{w,i}$.
- (2) Upon receiving $h_{w,j}$ for all $j \neq i$, $\mathcal{P}_i$ encrypts the randomness r_i with the encryption key pk_E as $ctx_{r,i} \leftarrow$

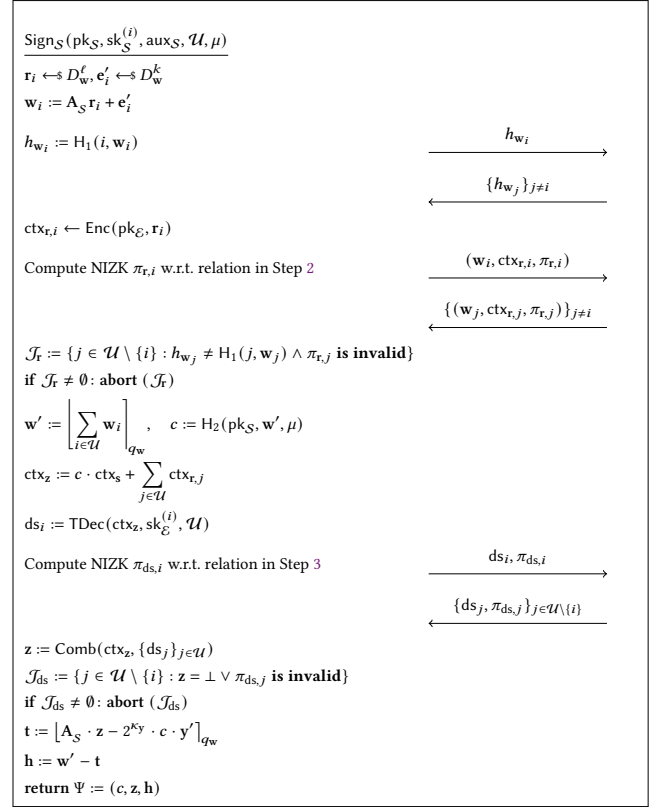


Figure 7: Actively secure 3-round t -out-of- n threshold signature protocol with bit dropping for party $\mathcal{P}_i$.

$\mathcal{E}.\text{Enc}(pk_E, r_i)$, and computes a proof $\pi_{r,i}$ to prove that r_i was used for $ctx_{r,i}$ and w_i . $\mathcal{P}_i$ then sends $(w_i, ctx_{r,i}, \pi_{r,i})$ to all $j \in \mathcal{U} \setminus \{i\}$.

- (3) Upon receiving $(w_j, ctx_{r,j}, \pi_{r,j})$ for each $j \in \mathcal{U} \setminus \{i\}$, $\mathcal{P}_i$ defines $\mathcal{J}_r := \{j \in \mathcal{U} \setminus \{i\} : h_{w,j} \neq H_1(j, w_j) \wedge \pi_{r,j} \text{ invalid w.r.t. } ctx_{r,j} \text{ and } w_j\}$. If $\mathcal{J}_r \neq \emptyset$, $\mathcal{P}_i$ aborts with output $\mathcal{J}_r$. Otherwise it computes $w := \sum_{i \in \mathcal{U}} w_i$, consequently $w' := \lfloor w \rfloor_{q_w}$ where $q_w = \lfloor q/2^{\kappa_w} \rfloor$, and computes the challenge $c := H_2(pk_S, w', \mu)$. It then computes the encryption of the signature as $ctx_z := c \cdot ctx_s + \sum_{j \in \mathcal{U}} ctx_{r,j}$ and decrypts its share as $ds_i \leftarrow \text{TDec}(ctx_z, sk_i, \mathcal{U})$ and sends ds_i to the parties in $\mathcal{U}$ alongside a proof $\pi_{ds,i}$ of correct decryption showing the correct key share is used.
- (4) Upon receiving ds_j for all $j \in \mathcal{U} \setminus \{i\}$, $\mathcal{P}_i$ combines the decryptions as $z := \text{Comb}(ctx_z, \{ds_j\}_{j \in \mathcal{U}})$ and defines $\mathcal{J}_{ds} := \{j \in \mathcal{U} \setminus \{i\} : z = \perp \vee \pi_{ds,j} \text{ invalid w.r.t. } ctx_z \text{ and } ds_j\}$. If $\mathcal{J}_{ds} \neq \emptyset$, $\mathcal{P}_i$ aborts with output $\mathcal{J}_{ds}$. $\mathcal{P}_i$ prepares hint $h := w' - t$ where $t := \lfloor A_S \cdot z - 2^{\kappa_y} \cdot c \cdot y' \rfloor_{q_w}$, and outputs the signature $\Psi := (c, z, h)$.

$\forall \mathcal{P}_i$: Upon receiving $\psi := (c, z, h)$ and μ , checks that $\|z, 2^{\kappa_w} \cdot h\| \leq B_z$, computes $w^* := \lfloor A_S \cdot z - 2^{\kappa_y} \cdot c \cdot y' \rfloor_{q_w} + h$, derives $c^* := H_2(pk_S, w^*, \mu)$, then finally outputs 1 if and only if checks hold and $c = c^*$, and 0 otherwise.

6 Security

Here we show correctness, unforgeability, and security with identifiable aborts for our signature scheme. The proofs are given in Appendices C.4, C.5, and C.6 respectively.

Theorem 6 (Correctness). *The threshold signature scheme described in Figure 7 is correct if $\kappa_w \geq 4$,*

$$B_{2,z} = e^{1/4} \cdot (nv\sigma_y + t\sigma_w) \sqrt{N(k+\ell)} + (v \cdot 2^{\kappa_y} + 2^{(\kappa_w+1)}) \cdot \sqrt{Nk}, \quad (2)$$

and the homomorphic threshold decryption correctness condition is satisfied.

Theorem 7 (Unforgeability). *Let the H-MLWE $_{k,\ell,\chi_1,\chi_2,Q}$ problem be $\epsilon_{\text{H-MLWE}}$ -hard. Let Q_S, Q_H denote the number of signing queries and the total queries made to H_0, H_1, H_2 respectively. Let ℓ_0, ℓ_1 be the bit-length of the output of H_0 and H_1 respectively. Let $\mathcal{E}$ be an ϵ_{IND} secure encryption scheme. Let $|C_U|$ denote the number of corrupted users. Let proof system Π_x for the relations $\mathcal{R}_x$ be $\epsilon_{\text{ZK},x}$, and $\epsilon_{\text{extract},x}$ -secure. Finally, let $\mathcal{B}$ be an adversary against the unforgeability of the underlying signature scheme. Then, the actively secure t -out-of- n threshold signature scheme $\mathcal{S}$, is ts-uf-cma-secure when H_0 and H_1 are modeled as programmable random oracles and H_2 is modeled as a random oracle. The advantage of adversary $\mathcal{A}$ is*

$$\begin{aligned} \text{Adv}_{\mathcal{S}}^{\text{ts-uf-cma}}(\mathcal{A}) \leq & \frac{|\mathcal{H}|(Q_H + Q_S)}{2^{\ell_0}} + \frac{|\mathcal{H}_U|(Q_H + Q_S)}{2^{\ell_1}} \\ & + |C| \cdot \epsilon_{\text{extract},s} + |C_U| \cdot \epsilon_{\text{extract},r} \\ & + |\mathcal{H}| \cdot \epsilon_{\text{ZK},s} + |\mathcal{H}_U| \cdot \epsilon_{\text{ZK},r} \\ & + \epsilon_{\text{IND}} + \epsilon_{\text{H-MLWE}} + \text{Adv}_{\mathcal{B}}^{\text{uf-cma}}(\mathcal{B}). \end{aligned}$$

REMARK. Note that while computation in the first two rounds of our protocol is independent of the message to be signed, our proof of Theorem 7 assumes the message is known to the simulator at the start of the second round. We also remark that if the message is included in the hash during the first round, it binds the adversary to the transcript and we achieve a concurrently secure protocol.

Theorem 8 (Security with Identifiable Aborts). *Let Pi_x be NIZKAoK corresponding to the relation $\mathcal{R}_x$. Let $\mathcal{E}$ be a correct linear homomorphic encryption scheme with identifiable aborts. Let H_0, H_1 be hash functions modeled as random oracles. Then the actively secure threshold signature scheme $\mathcal{S}$ has security with identifiable aborts in the synchronous model with access to a broadcast channel.*

7 Performance

7.1 Parameters and Sizes

In Table 2 we collect the parameters governing our scheme and the constraints that apply to them. Table 3 displays the concrete values that satisfy each of the three security levels.

In Table 4 we present the communication costs of Olingo's signing procedure as compared to TRaccoon [30] and Ringtail [17]. Total communication includes all rounds and the online communication refers to messages sent during message-dependent rounds. Our final NIZK proof $\pi_{\text{ds},i}$ is only sent if the final signature fails to verify in the optimistic setting, where we assume that most signature executions go well. See Table 1 for a comparison of these sizes to a wider list of existing works; we choose TRaccoon and Ringtail here since they currently are the only implemented schemes.

Table 2: Parameters of the signature scheme

Parameter	Description	Constraints (if any)
(t, n)	# of threshold signing parties and total # of signing parties	$t \leq n$
N	Dimension of polynomial ring	$\text{poly}(\lambda)$
(k, ℓ)	Dimensions of public signing matrix A_s	Set so that MLWE and MSIS hard
$(\hat{k}, \hat{\ell})$	Dimensions of public encryption matrix A_e	Set so that MLWE and MSIS hard
Q	max # of signatures issued	$Q \approx 2^{60}$
(κ_y, κ_w)	# of bits dropped for verification key y and the commitment w	$O(\log \lambda), \kappa_w \geq 4$
v	ℓ_1 norm bound for the signature challenge c from C_v	$ C_v \geq 2^\lambda$
$B_{\text{H-MLWE}}$	Bound required for the H-MLWE to MLWE reduction	$B_{\text{H-MLWE}} = Q \cdot v \cdot \left(1 + \frac{N(\lambda+1+2\log(N))}{\sqrt{Q}}\right)$
σ_{LWE}	Gaussian width of encryption secret keys and encryption randomness	Set so that MLWE and MSIS are hard
σ_y	Gaussian width of user signing key secrets in y	$\sigma_y = 2\sqrt{\ell + k} \cdot \log N$
σ_w	Gaussian width of the signature randomness in w	$\sigma_w = 2\sqrt{B_{\text{H-MLWE}}} \cdot (\ell + k) \cdot \log N$
σ	Gaussian width for the H-MLWE to MLWE reduction	$\sigma \geq \sqrt{\ell + k} \cdot \omega(\sqrt{\log N})$ $\frac{1}{\sigma^2} = 2 \cdot \left(\frac{1}{n \cdot \sigma_y^2} + \frac{B_{\text{H-MLWE}}}{t \cdot \sigma_w^2}\right)$
$B_{2,z}$	Bound for the signature z	$B_{2,z} = e^{1/4} \cdot (nv\sigma_y + t\sigma_w) \sqrt{N(k+\ell)} + (v \cdot 2^{\kappa_y} + 2^{(\kappa_w+1)}) \cdot \sqrt{Nk}$
$B_{\text{ST-MSIS}}$	Bound required for the ST-MSIS security	$B_{\text{ST-MSIS}} = B_{2,z} + \sqrt{v} + (v \cdot 2^{\kappa_y} + 2^{(\kappa_w+1)}) \cdot \sqrt{Nk}$
B_{MSIS}	Bound required for the ST-MSIS to MSIS reduction	$B_{\text{MSIS}} = 2B_{\text{ST-MSIS}}$
q	Signature modulus	$q > B_{\text{MSIS}} \sqrt{Nk} \cdot \omega(\log(Nk))$

Table 3: Parameter values for Olingo for thresholds up to $(t, n) = (1024, 1024)$ and a maximum of $Q = 2^{60}$ signatures. When $\sigma_{\text{LWE}} = \sqrt{2/3}$ we sample from a binary distribution.

Param	λ	N	q	$\hat{q}$	ℓ	$\hat{\ell}$	k	$\hat{k}$	κ_y	κ_w	v	σ_{LWE}	σ_y	σ_w
Level I	128	256	2^{46}	2^{114}	8	21	8	21	29	32	23	$\sqrt{2/3}$	2^{10}	2^{29}
Level II	192	256	2^{47}	2^{116}	11	29	11	29	29	33	39	$\sqrt{2/3}$	2^{10}	2^{30}
Level III	256	256	2^{48}	2^{118}	14	37	14	37	29	33	60	$\sqrt{2/3}$	2^{10}	2^{30}

We used the *lattice-estimator*¹⁰ by Albrecht et al. [2] to estimate the security of our parameters. We remark that binary secrets are commonly used for encryption schemes based on lattices [19], providing the same security with respect to the learning with errors assumption as for other distributions [62], also ensuring security against the Arora-Ge combinatorial attack [5] for up to 2^{60} samples in the module setting.

7.2 Implementation and Benchmarks

Our implementation recycles components for polynomial multiplication and NTT from the Dilithium C implementation¹¹ and the discrete Gaussian sampler¹² by Zhao, Steinfeld and Sakzad [74]. Moreover we use the LaZer Library¹³ [60] for the execution of NIZK

¹⁰Lattice-estimator: <https://github.com/malb/lattice-estimator>

¹¹Dilithium: <https://github.com/pq-crystals/dilithium>

¹²Gaussian sampler: <https://gitlab.com/raykzhao/gaussian>

¹³LaZer: <https://github.com/lazer-crypto/lazer>

Table 4: Comparison with TRaccoon [30] and Ringtail [17]. Sizes are given in KB per signer at $(t, n) = (1024, 1024)$ threshold. Communication costs are for total and online communication. The optimistic communication is without NIZKs in the online phase, which is only required for identifiable abort. Note, with the use of LaBRADOR, the online phase of Olingo can be reduced to only the optimistic costs + 66KB.

Scheme	λ	pk_S	Ψ	Total COM	Online COM (Optimistic)
TRaccoon	128	3.9	12.7	41	41
Ringtail	128	4.5	13.4	613	11
Olingo	128	4.4	11.3	852	563 (76)
TRaccoon	192	5.8	18.9	59	59
Ringtail	192	6.4	19.9	–	–
Olingo	192	6.4	15.8	1254	860 (108)
TRaccoon	256	7.2	21.6	69	69
Ringtail	256	8.5	27.3	–	–
Olingo	256	8.4	20.1	1702	1208 (139)

proofs of exact norms. We adapt the linearity proof used in [4]¹⁴ for our linearity proofs. We have modified these components to be compatible with larger integers using the Gnu Multiple Precision library [47]. We will make our implementation available on GitHub.

Benchmarks are run on a desktop computer with an Intel i9-14900 CPU, 64 GB RAM running Ubuntu 24.04.2. The timings for the DKG are averaged over 10 runs (except for $KGen_E$ using $n = 128$). The timings for signing and verification are averaged over 10 runs, and the benchmarks of the NIZKs are also averaged over 10 runs. To simplify our analysis, where possible, we simulate only one user and ignore all potential communication latency¹⁵.

We chose to use the LNP proofs [59] of the LaZer library [60] for implementing the zero-knowledge proofs, as it is, at the time of writing, the easiest to use and, for many use cases, the most efficient available library for proving exact norms over lattices with online extractability. We note that the authors claim a ~30% overhead when using the Python wrapper for their library. In addition, the LaZer library requires that we decompose vectors with ring elements whose coefficients are sampled from a large Gaussian, since it is not specifically designed for our parameters, leading to slower proof generation and verification times and a larger proof size in the final signing round in $Sign_S$. In a real-world implementation, these proofs would likely be faster than showcased in our benchmarks.

We would have preferred to use LaBRADOR for the final signing-round proof, where online extractability is not needed, owing to its succinctness (66 KB). However, the LaZer codebase does not currently support parameters of our magnitude, and we use LNP instead for the benchmarks. The linearity proofs used in $KGen_E$ and in the final signing round of $Sign_S$ use the Π_{LIN} proof from [9].

Table 5 shows the running time of our distributed key generation $KGen_S$ for various thresholds $(n, t = n - 1)$. The first column

Table 5: Timing provided in seconds for our distributed signing key generation $KGen_S$. The timings are given as a sum; the second component represents the time taken to generate the relevant NIZKs, and the first component represents the time to run the rest of the protocol.

n	$KGen_E$	$KGen_S$ (total)	$KGen_S$ (verify)
16	19.45 = 10.60 + 8.85	21.42 = 11.10 + 10.32	17.30
32	43.32 = 33.86 + 9.46	45.29 = 34.36 + 10.93	41.37
64	172.08 = 161.34 + 10.74	174.05 = 161.84 + 12.21	107.60
128	1494.14 = 1480.74 + 13.40	1496.11 = 1481.24 + 14.87	317.15

indicates the time to run the distributed encryption key generation $KGen_E$, where we provide both the time to run the (passively secure) base protocol and the time to generate the NIZKs needed for active security. In the second columns we do the same for the whole distributed signing key generation $KGen_S$ (which internally runs $KGen_E$). The final column shows the time to verify $KGen_S$, i.e., the various NIZKs produced throughout the protocol. To obtain these benchmarks, we focus only on the running time of a single party, assuming that all offline computations performed by other parties run in parallel. To further avoid unnecessary memory usage, we simulate the inputs from other parties using ‘dummy’ objects. We note that these are chosen to reflect the cost of computing on ‘real’ inputs. We observe that, whilst the key generation could not be called fast, the timings are perfectly acceptable for a one-time setup computation. Indeed, even up to 64 signers, the total key generation can be done in under 3 minutes. Table 6 shows the communication in the distributed key generation protocol.

Table 7 shows the signing times of the main signing protocol $Sign$. We have split the timings of $Sign_S$ into three parts: $Sign_{S_1}$, $Sign_{S_2}$ and $Sign_{S_3}$, corresponding to rounds 1, 2 and 3 in Figure 7. The numbers for $Sign_{S_2}$ are presented as a sum, with the second and third terms representing proof generation and proof verification, respectively. We emphasize that for real-world thresholds (small n and t), the t -dependence in verification would likely be acceptable, and per-proof verification would be smaller owing to the LaZer Python wrapper overhead. Furthermore, this verification could be parallelized. We note that only $Sign_{S_3}$ is considered to be an ‘online’ round, and the NIZK in that round needs only to be sent if there is a dispute. In the case of such a dispute, our benchmarks show a verification time of roughly $700 \cdot t$ ms, which again, could likely be reduced in practice via parallelization and a clean LNP implementation. We include, for comparison, the timings for Threshold Raccoon [30] and Ringtail [17]. We wish to emphasize that neither Threshold Raccoon nor Ringtail offers a distributed key generation or achieves identifiable aborts. Whilst the addition of these makes our design very attractive from a security perspective, Table 7 demonstrates its efficiency comparable to those less fully-featured designs.

7.3 Non-Interactive Signing

Our signing protocol, depicted in Figure 7, has three rounds of communication, compared to the two rounds of [48]. This increase in the number of rounds comes with an advantage, as the first two rounds of our protocol are independent of the message to be signed.

¹⁴Proofs of Linearity: <https://github.com/dfaranha/lattice-verifiable-mixnet>

¹⁵We exclude the broadcast channel from all experiments and focus solely on computational overhead.

Table 6: Communication cost given in MB. The cost is given as a sum; the second component represents the cost of the relevant NIZKs and the first represents the cost of the rest of the protocol. These values are computed for $(n, t) = (1024, 1023)$ and are thus upper bounds on the communication cost at lower thresholds.

λ	KGen $_{\mathcal{E}}$	KGen $_{\mathcal{S}}$ (total)
128	2003.5 = 2002.1 + 1.4	2003.8 = 2002.3 + 1.5
192	3910.7 = 3908.6 + 2.1	3911.1 = 3908.8 + 2.3
256	6439.3 = 6436.2 + 3.1	6439.7 = 6436.4 + 3.3

Table 7: Signing timings for our scheme, TRaccoon [30] and Ringtail [17]. All timings are listed in milliseconds, computed by dividing the number of CPU cycles by the CPU frequency. We did not include the time needed to allocate space on the heap for shared variables. The summations in rounds 2 and 3 timings reflect the additional cost of generating the NIZKs and thus achieving IA security. Note that all timings are set according to the same parameter set, which ensures 128-bit security up to the maximum thresholds $(n, t) = (1024, 1023)$; in practice, timings are likely to be faster for smaller thresholds.

Scheme	t	Sign $_{S_1}$	Sign $_{S_2}$	Sign $_{S_3}$	Comb	Verify
Olingo	4	1.304	22.544 + 1682.00 + 703 · t	2.840 + 1365.61	0.810	1.046
Olingo	16	1.304	22.544 + 1682.00 + 879 · t	3.920 + 1365.61	1.030	1.046
Olingo	64	1.304	22.544 + 1682.00 + 923 · t	8.280 + 1365.61	1.820	1.046
Olingo	256	1.304	22.544 + 1682.00 + 934 · t	25.020 + 1365.61	4.940	1.046
Olingo	1023	1.304	22.544 + 1682.00 + 936 · t	92.790 + 1365.61	17.500	1.046
TRaccoon	4	20.092	0.539	1.588	1.128	1.094
TRaccoon	16	20.076	2.102	5.559	1.209	1.093
TRaccoon	64	21.830	8.216	21.350	1.579	1.100
TRaccoon	256	33.549	32.788	84.333	3.186	1.095
TRaccoon	1024	67.213	131.887	338.614	11.571	1.106
Ringtail	4	18.995	131.944	4.601	0.143	1.361
Ringtail	16	29.084	164.646	13.904	0.179	1.331
Ringtail	64	73.684	313.938	52.036	0.339	1.627
Ringtail	256	254.253	829.033	201.272	0.821	1.515
Ringtail	1024	978.182	2949.853	794.288	2.706	1.434

Hence, it is possible to bootstrap our scheme to a non-interactive threshold signature scheme.

Instead of running the first two rounds of signing in an interactive manner, each $\mathcal{U}$ set can run a “presigning” round to determine $\mathbf{w}^{(\tau)}$, $\text{ctx}_{\mathbf{r}}^{(\tau)}$ to use in τ -th time they has to sign a message. Then whenever $\mathcal{U}$ has to sign $\mu^{(\tau)}$, $\mathcal{U}$ can use $\mathbf{w}^{(\tau)}$ to derive $c^{(\tau)}$ and consecutively $\text{ctx}_{\mathbf{r}}^{(\tau)}$.

With pre-signing, there is only one round of communication, which then includes the partial decryption shares ds_i . However, this also means that the final round of communication can be avoided. Combining decryption shares does not require interaction; hence, each decryption share can be published individually for an external

party to combine. This makes the protocol with presigning *non-interactive*¹⁶.

Another advantage of non-interactivity is the capability of batching NIZKs. The first two rounds of signing that can be pre-processed contain a proof of plaintext knowledge for signature randomness $\mathbf{r}$. While the value of $\mathbf{r}$ and consequently $\mathbf{w}$ changes for different signing queries, $\mathbf{A}_{\mathcal{S}}$ remains the same. Hence, instead of sending Q proofs per party in the first round, we can send a single proof of increased witness size. Considering we need to prove a set of inner products, we can use LaBRADOR [12] that allows succinct proofs for inner product relations in the ring $R'_q = \mathbb{Z}_q[X]/(X^{64} + 1)$. This way $\pi_{\mathbf{r}}$ for Q values of $\mathbf{r}$ constitutes a single proof of an increased witness size of $\lceil N/64 \rceil \cdot (Q \cdot \ell)$.

7.4 Optimistic Approach

In addition to making the scheme non-interactive, we can utilize the “optimistic” approach taken in [48] to reduce communication bandwidth. Instead of sending a NIZK alongside ds_i , each signer only sends ds_i . If the combined signature verifies, the protocol is completed. If not, each signer can ask for the corresponding NIZKs to verify that each ds_i was correctly computed. In an optimistic scenario where the final signature verifies, this reduces the final communication size to 76 KB.

TRaccoon-IA [66] also provides an optimistic approach for their threshold signature scheme reducing their communication cost to 28.2 KB, independent of the threshold. This is less than half of our optimistic communication cost. However, Olingo provides a distributed key generation which is not provided in [66]. Both Olingo’s optimistic approach and that of [66] would require an additional round of communication to send the NIZKs in case of a dispute to achieve identifiable aborts, narrowing the gap in communication costs between the two schemes for larger thresholds.

Acknowledgments

We thank Patrick Steuer and Vadim Lyubashevsky for help with the LaZer library, Diego F. Aranha for input on optimizing the implementation, and Ignacio Cascudo for pointers on improving the verifiable secret sharing scheme. We also thank Sascha Roth and Matthieu Rambaud for discussions on the security model and Guilhem Niot for input regarding the parameter sets.

¹⁶Again, we assume that the participating set of users $\mathcal{U}$ is known a priori.

References

- [1] Shweta Agrawal, Damien Stehlé, and Anshu Yadav. 2022. Round-Optimal Lattice-Based Threshold Signatures, Revisited. In *49th International Colloquium on Automata, Languages, and Programming (ICALP 2022) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 229)*, Mikołaj Bojańczyk, Emanuela Merelli, and David P. Woodruff (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 8:1–8:20. doi:10.4230/LIPIcs.ICALP.2022.8
- [2] Martin R. Albrecht, Rachel Player, and Sam Scott. 2015. On the concrete hardness of Learning with Errors. *Journal of Mathematical Cryptology* 9, 3 (2015), 169–203. doi:10.1515/jmc-2015-0016
- [3] Nabil Alkailani Alkadri, Nico Döttling, and Sihang Pu. 2024. Practical Lattice-Based Distributed Signatures for a Small Number of Signers. In *ACNS 2024, Part I (LNCS, Vol. 14583)*, Christina Pöpper and Lejla Batina (Eds.). Springer, Cham, Switzerland, Abu Dhabi, UAE, 376–402. doi:10.1007/978-3-031-54770-6_15
- [4] Diego F. Aranha, Carsten Baum, Kristian Gjøsteen, and Tjerdan Silde. 2023. Verifiable Mix-Nets and Distributed Decryption for Voting from Lattice-Based Assumptions. In *ACM CCS 2023*, Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda (Eds.). ACM Press, Copenhagen, Denmark, 1467–1481. doi:10.1145/3576915.3616683
- [5] Sanjeev Arora and Rong Ge. 2011. New Algorithms for Learning in Presence of Errors. In *ICALP 2011, Part I (LNCS, Vol. 6755)*, Luca Aceto, Monika Henzinger, and Jiri Sgall (Eds.). Springer Berlin Heidelberg, Germany, Zurich, Switzerland, 403–415. doi:10.1007/978-3-642-22006-7_34
- [6] Renas Bacho and Julian Loss. 2022. On the Adaptive Security of the Threshold BLS Signature Scheme. In *ACM CCS 2022*, Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi (Eds.). ACM Press, Los Angeles, CA, USA, 193–207. doi:10.1145/3548606.3560656
- [7] Shi Bai and Steven D. Galbraith. 2014. An Improved Compression Technique for Signatures Based on Learning with Errors. In *CT-RSA 2014 (LNCS, Vol. 8366)*, Josh Benaloh (Ed.). Springer, Cham, Switzerland, San Francisco, CA, USA, 28–47. doi:10.1007/978-3-319-04852-9_2
- [8] Shi Bai, Tancrede Lepoint, Adeline Roux-Langlois, Amin Sakzad, Damien Stehlé, and Ron Steinfeld. 2018. Improved Security Proofs in Lattice-Based Cryptography: Using the Rényi Divergence Rather than the Statistical Distance. *Journal of Cryptology* 31, 2 (April 2018), 610–640. doi:10.1007/s00145-017-9265-9
- [9] Carsten Baum, Ivan Damgård, Vadim Lyubashevsky, Sabine Oechsner, and Chris Peikert. 2018. More Efficient Commitments from Structured Lattice Assumptions. In *SCN 18 (LNCS, Vol. 11035)*, Dario Catalano and Roberto De Prisco (Eds.). Springer, Cham, Switzerland, Amalfi, Italy, 368–385. doi:10.1007/978-3-319-98113-0_20
- [10] Josh Cohen Benaloh and Jerry Leichter. 1990. Generalized Secret Sharing and Monotone Functions. In *CRYPTO’88 (LNCS, Vol. 403)*, Shafi Goldwasser (Ed.). Springer, New York, USA, Santa Barbara, CA, USA, 27–35. doi:10.1007/0-387-34799-2_3
- [11] Rikke Bendlin, Sara Krehbiel, and Chris Peikert. 2013. How to Share a Lattice Trapdoor: Threshold Protocols for Signatures and (H)IBE. In *ACNS 2013 (LNCS, Vol. 7954)*, Michael J. Jacobson, Jr., Michael E. Locasto, Payman Mohassel, and Reihaneh Safavi-Naini (Eds.). Springer Berlin Heidelberg, Germany, Banff, AB, Canada, 218–236. doi:10.1007/978-3-642-38980-1_14
- [12] Ward Beullens and Gregor Seiler. 2023. LaBRADOR: Compact Proofs for R1CS from Module-SIS. In *CRYPTO 2023, Part V (LNCS, Vol. 14085)*, Helena Handschuh and Anna Lysyanskaya (Eds.). Springer, Cham, Switzerland, Santa Barbara, CA, USA, 518–548. doi:10.1007/978-3-031-38554-4_17
- [13] Alexander Bienstock, Leo de Castro, Daniel Escudero, Antigoni Polychroniadou, and Akira Takahashi. 2025. Efficient, Scalable Threshold ML-DSA Signatures: An MPC Approach. *Cryptology ePrint Archive, Report 2025/1163*. <https://eprint.iacr.org/2025/1163>
- [14] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. 2018. Threshold Cryptosystems from Threshold Fully Homomorphic Encryption. In *CRYPTO 2018, Part I (LNCS, Vol. 10991)*, Hovav Shacham and Alexandra Boldyreva (Eds.). Springer, Cham, Switzerland, Santa Barbara, CA, USA, 565–596. doi:10.1007/978-3-319-96884-1_19
- [15] Jonathan Bootle, Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Alessandro Sorniotti. 2023. A Framework for Practical Anonymous Credentials from Lattices. In *CRYPTO 2023, Part II (LNCS, Vol. 14082)*, Helena Handschuh and Anna Lysyanskaya (Eds.). Springer, Cham, Switzerland, Santa Barbara, CA, USA, 384–417. doi:10.1007/978-3-031-38545-2_13
- [16] Giacomo Borin, Sofia Celi, Rafaél del Pino, Thomas Espitau, Guilhem Niot, and Thomas Prest. 2025. Threshold Signatures Reloaded: ML-DSA and Enhanced Raccoon with Identifiable Aborts. *Cryptology ePrint Archive, Report 2025/1166*. <https://eprint.iacr.org/2025/1166>
- [17] Cecilia Boschini, Darya Kaviani, Russell W. F. Lai, Giulio Malavolta, Akira Takahashi, and Mehdi Tibouchi. 2025. Ringtail: Practical Two-Round Threshold Signatures from Learning with Errors. In *2025 IEEE Symposium on Security and Privacy*, Marina Blanton, William Enck, and Cristina Nita-Rotaru (Eds.). IEEE Computer Society Press, San Francisco, CA, USA, 149–164. doi:10.1109/SP61157.2025.00070
- [18] Cecilia Boschini, Akira Takahashi, and Mehdi Tibouchi. 2022. MuSig-L: Lattice-Based Multi-signature with Single-Round Online Phase. In *CRYPTO 2022, Part II (LNCS, Vol. 13508)*, Yevgeniy Dodis and Thomas Shrimpton (Eds.). Springer, Cham, Switzerland, Santa Barbara, CA, USA, 276–305. doi:10.1007/978-3-031-15979-4_10
- [19] Jean-Philippe Bossuat, Rosario Cammarota, Ilaria Chillotti, Benjamin R. Curtis, Wei Dai, Huijing Gong, Erin Hales, Duhyeon Kim, Bryan Kumara, Changmin Lee, Xianhui Lu, Carsten Maple, Alberto Pedrouzo-Ulloa, Rachel Player, Yuriy Polyakov, Luis Antonio Ruiz Lopez, Yongsoo Song, and Donggeon Yhee. 2024. Security Guidelines for Implementing Homomorphic Encryption. *CiC 1*, 4 (2024), 26. doi:10.62056/anxra69p1
- [20] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. 2012. (Leveled) fully homomorphic encryption without bootstrapping. In *ITCS 2012*, Shafi Goldwasser (Ed.). ACM, Cambridge, MA, USA, 309–325. doi:10.1145/2090236.2090262
- [21] Luis T. A. N. Brandão and René Peralta. 2025. NIST First Call for Multi-Party Threshold Schemes. NIST Internal Report (NISTIR) 8214C 2pd (Second Public Draft). <https://doi.org/10.6028/NIST.IR.8214C.2pd>
- [22] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. 2018. Bulletproofs: Short Proofs for Confidential Transactions and More. In *2018 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, San Francisco, CA, USA, 315–334. doi:10.1109/SP.2018.00020
- [23] Ignacio Cascudo and Bernardo David. 2017. SCRAPE: Scalable Randomness Attested by Public Entities. In *ACNS 2017 (LNCS, Vol. 10355)*, Dieter Gollmann, Atsuko Miyaji, and Hiroaki Kikuchi (Eds.). Springer, Cham, Switzerland, Kanazawa, Japan, 537–556. doi:10.1007/978-3-319-61204-1_27
- [24] Rutuchon Chairattana-Apirom, Stefano Tessaro, and Chenzhi Zhu. 2024. Partially Non-interactive Two-Round Lattice-Based Threshold Signatures. In *ASIACRYPT 2024, Part IV (LNCS, Vol. 15487)*, Kai-Min Chung and Yu Sasaki (Eds.). Springer, Singapore, Singapore, Kolkata, India, 268–302. doi:10.1007/978-981-96-0894-2_9
- [25] Yanbo Chen. 2023. DualMS: Efficient Lattice-Based Two-Round Multi-signature with Trapdoor-Free Simulation. In *CRYPTO 2023, Part V (LNCS, Vol. 14085)*, Helena Handschuh and Anna Lysyanskaya (Eds.). Springer, Cham, Switzerland, Santa Barbara, CA, USA, 716–747. doi:10.1007/978-3-031-38554-4_23
- [26] Benny Chor, Shafi Goldwasser, Silvio Micali, and Baruch Awerbuch. 1985. Verifiable Secret Sharing and Achieving Simultaneity in the Presence of Faults (Extended Abstract). In *26th FOCS*. IEEE Computer Society Press, Portland, Oregon, 383–395. doi:10.1109/SFCS.1985.64
- [27] Daniele Cozzo and Nigel P. Smart. 2019. Sharing the LUOV: Threshold Post-quantum Signatures. In *17th IMA International Conference on Cryptography and Coding (LNCS, Vol. 11929)*, Martin Albrecht (Ed.). Springer, Cham, Switzerland, Oxford, UK, 128–153. doi:10.1007/978-3-030-35199-1_7
- [28] Ivan Damgård, Claudio Orlandi, Akira Takahashi, and Mehdi Tibouchi. 2021. Two-Round n-out-of-n and Multi-signatures and Trapdoor Commitment from Lattices. In *PKC 2021, Part I (LNCS, Vol. 12710)*, Juan Garay (Ed.). Springer, Cham, Switzerland, Virtual Event, 99–130. doi:10.1007/978-3-030-75245-3_5
- [29] Rafaél del Pino, Thomas Espitau, Guilhem Niot, and Thomas Prest. 2025. Simple and Efficient Lattice Threshold Signatures with Identifiable Aborts. *Cryptology ePrint Archive, Report 2025/871*. <https://eprint.iacr.org/2025/871>
- [30] Rafaél del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani O. Saarinen. 2024. Threshold Raccoon: Practical Threshold Signatures from Standard Lattice Assumptions. In *EUROCRYPT 2024, Part II (LNCS, Vol. 14652)*, Marc Joye and Gregor Leander (Eds.). Springer, Cham, Switzerland, Zurich, Switzerland, 219–248. doi:10.1007/978-3-031-58723-8_8
- [31] Rafaél del Pino, Shuichi Katsumata, Thomas Prest, and Mélissa Rossi. 2024. Raccoon: A Masking-Friendly Signature Proven in the Probing Model. In *CRYPTO 2024, Part I (LNCS, Vol. 14920)*, Leonid Reyzin and Douglas Stebila (Eds.). Springer, Cham, Switzerland, Santa Barbara, CA, USA, 409–444. doi:10.1007/978-3-031-68376-3_13
- [32] Yvo Desmedt. 1988. Society and Group Oriented Cryptography: A New Concept. In *CRYPTO’87 (LNCS, Vol. 293)*, Carl Pomerance (Ed.). Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA, 120–127. doi:10.1007/3-540-48184-2_8
- [33] Yvo Desmedt and Yair Frankel. 1990. Threshold Cryptosystems. In *CRYPTO’89 (LNCS, Vol. 435)*, Gilles Brassard (Ed.). Springer, New York, USA, Santa Barbara, CA, USA, 307–315. doi:10.1007/0-387-34805-0_28
- [34] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. 2024. Threshold ECDSA in Three Rounds. In *2024 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, San Francisco, CA, USA, 3053–3071. doi:10.1109/SP54263.2024.00178
- [35] Manu Drijvers, Kasra Edalatnejad, Bryan Ford, Eike Kiltz, Julian Loss, Gregory Neven, and Igors Stepanovs. 2019. On the Security of Two-Round Multi-Signatures. In *2019 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, San Francisco, CA, USA, 1084–1101. doi:10.1109/SP.2019.00050
- [36] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. 2018. CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme. *IACR TCHES* 2018, 1 (2018), 238–268. doi:10.13154/tches.v2018.i1.238-268
- [37] Thomas Espitau, Shuichi Katsumata, and Kaoru Takemure. 2024. Two-Round Threshold Signature from Algebraic One-More Learning with Errors.

- In *CRYPTO 2024, Part VII (LNCS, Vol. 14926)*, Leonid Reyzin and Douglas Stebila (Eds.). Springer, Cham, Switzerland, Santa Barbara, CA, USA, 387–424. doi:10.1007/978-3-031-68394-7_13
- [38] Thomas Espitau, Guilhem Niot, and Thomas Prest. 2024. Flood and Submerge: Distributed Key Generation and Robust Threshold Signature from Lattices. In *CRYPTO 2024, Part VII (LNCS, Vol. 14926)*, Leonid Reyzin and Douglas Stebila (Eds.). Springer, Cham, Switzerland, Santa Barbara, CA, USA, 425–458. doi:10.1007/978-3-031-68394-7_14
- [39] Paul Feldman. 1987. A Practical Scheme for Non-interactive Verifiable Secret Sharing. In *28th FOCS*. IEEE Computer Society Press, Los Angeles, CA, USA, 427–437. doi:10.1109/SFCS.1987.4
- [40] Amos Fiat and Adi Shamir. 1987. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *CRYPTO’86 (LNCS, Vol. 263)*, Andrew M. Odlyzko (Ed.). Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA, 186–194. doi:10.1007/3-540-47721-7_12
- [41] Nils Fleischhacker, Gottfried Herold, Mark Simkin, and Zhenfei Zhang. 2023. Chipmunk: Better Synchronized Multi-Signatures from Lattices. In *ACM CCS 2023*, Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda (Eds.). ACM Press, Copenhagen, Denmark, 386–400. doi:10.1145/3576915.3623219
- [42] Nils Fleischhacker, Mark Simkin, and Zhenfei Zhang. 2022. Squirrel: Efficient Synchronized Multi-Signatures from Lattices. In *ACM CCS 2022*, Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi (Eds.). ACM Press, Los Angeles, CA, USA, 1109–1123. doi:10.1145/3548606.3560655
- [43] Masayuki Fukumitsu and Shingo Hasegawa. 2020. A Lattice-Based Provably Secure Multisignature Scheme in Quantum Random Oracle Model. In *ProvSec 2020 (LNCS, Vol. 12505)*, Khoa Nguyen, Wenling Wu, Kwok-Yan Lam, and Huaxiong Wang (Eds.). Springer, Cham, Switzerland, Singapore, 45–64. doi:10.1007/978-3-030-62576-4_3
- [44] Rosario Gennaro, Michael O. Rabin, and Tal Rabin. 1998. Simplified VSS and Fast-Track Multiparty Computations with Applications to Threshold Cryptography. In *17th ACM PODC*, Brian A. Coan and Yehuda Afek (Eds.). ACM, Puerto Vallarta, Mexico, 101–111. doi:10.1145/277697.277716
- [45] Craig Gentry, Shai Halevi, and Vadim Lyubashevsky. 2022. Practical Non-interactive Publicly Verifiable Secret Sharing with Thousands of Parties. In *EUROCRYPT 2022, Part I (LNCS, Vol. 13275)*, Orr Dunkelman and Stefan Dziembowski (Eds.). Springer, Cham, Switzerland, Trondheim, Norway, 458–487. doi:10.1007/978-3-031-06944-4_16
- [46] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. 2008. Trapdoors for hard lattices and new cryptographic constructions. In *40th ACM STOC*, Richard E. Ladner and Cynthia Dwork (Eds.). ACM Press, Victoria, BC, Canada, 197–206. doi:10.1145/1374376.1374407
- [47] Torbjörn Granlund and the GMP development team. 2012. *GNU MP: The GNU Multiple Precision Arithmetic Library* (5.0.5 ed.). <http://gmplib.org/>.
- [48] Kamil Doruk Gur, Jonathan Katz, and Tjerand Silde. 2024. Two-Round Threshold Lattice-Based Signatures from Threshold Homomorphic Encryption. In *Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part II*, Markku-Juhani Saareinen and Daniel Smith-Tone (Eds.). Springer, Cham, Switzerland, Oxford, UK, 266–300. doi:10.1007/978-3-031-62746-0_12
- [49] Yuval Ishai, Rafail Ostrovsky, and Vassilis Zikas. 2014. Secure Multi-Party Computation with Identifiable Abort. In *CRYPTO 2014, Part II (LNCS, Vol. 8617)*, Juan A. Garay and Rosario Gennaro (Eds.). Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA, 369–386. doi:10.1007/978-3-662-44381-1_21
- [50] Shuichi Katsumata. 2021. A New Simple Technique to Bootstrap Various Lattice Zero-Knowledge Proofs to QROM Secure NIZKs. In *CRYPTO 2021, Part II (LNCS, Vol. 12826)*, Tal Malkin and Chris Peikert (Eds.). Springer, Cham, Switzerland, Virtual Event, 580–610. doi:10.1007/978-3-030-84245-1_20
- [51] Shuichi Katsumata, Michael Reichle, and Kaoru Takemure. 2024. Adaptively Secure 5 Round Threshold Signatures from MLWE/MSIS and DL with Rewinding. In *CRYPTO 2024, Part VII (LNCS, Vol. 14926)*, Leonid Reyzin and Douglas Stebila (Eds.). Springer, Cham, Switzerland, Santa Barbara, CA, USA, 459–491. doi:10.1007/978-3-031-68394-7_15
- [52] Jonathan Katz, Chiu-Yuen Koo, and Ranjit Kumaresan. 2008. Improving the Round Complexity of VSS in Point-to-Point Networks. In *ICALP 2008, Part II (LNCS, Vol. 5126)*, Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfssdóttir, and Igor Walukiewicz (Eds.). Springer Berlin Heidelberg, Germany, Reykjavik, Iceland, 499–510. doi:10.1007/978-3-540-70583-3_41
- [53] Duhyeon Kim, Dongwon Lee, Jinyeong Seo, and Yongsoo Song. 2023. Toward Practical Lattice-Based Proof of Knowledge from Hint-MLWE. In *CRYPTO 2023, Part V (LNCS, Vol. 14085)*, Helena Handschuh and Anna Lysyanskaya (Eds.). Springer, Cham, Switzerland, Santa Barbara, CA, USA, 549–580. doi:10.1007/978-3-031-38554-4_18
- [54] Chelsea Komlo and Ian Goldberg. 2020. FROST: Flexible Round-Optimized Schnorr Threshold Signatures. In *SAC 2020 (LNCS, Vol. 12804)*, Orr Dunkelman, Michael J. Jacobson, Jr., and Colin O’Flynn (Eds.). Springer, Cham, Switzerland, Halifax, NS, Canada (Virtual Event), 34–65. doi:10.1007/978-3-030-81652-0_2
- [55] Adeline Langlois and Damien Stehlé. 2015. Worst-case to average-case reductions for module lattices. *DCC* 75, 3 (2015), 565–599. doi:10.1007/s10623-014-9938-4
- [56] Adeline Langlois, Damien Stehlé, and Ron Steinfeld. 2014. GGHLite: More Efficient Multilinear Maps from Ideal Lattices. In *EUROCRYPT 2014 (LNCS, Vol. 8441)*, Phong Q. Nguyen and Elisabeth Oswald (Eds.). Springer Berlin Heidelberg, Germany, Copenhagen, Denmark, 239–256. doi:10.1007/978-3-642-55220-5_14
- [57] Yehuda Lindell. 2017. Fast Secure Two-Party ECDSA Signing. In *CRYPTO 2017, Part II (LNCS, Vol. 10402)*, Jonathan Katz and Hovav Shacham (Eds.). Springer, Cham, Switzerland, Santa Barbara, CA, USA, 613–644. doi:10.1007/978-3-319-63715-0_21
- [58] Vadim Lyubashevsky, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Peter Schwabe, Gregor Seiler, Damien Stehlé, and Shi Bai. 2022. *CRYSTALS-DILITHIUM*. Technical Report. National Institute of Standards and Technology. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- [59] Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Maxime Plançon. 2022. Lattice-Based Zero-Knowledge Proofs and Applications: Shorter, Simpler, and More General. In *CRYPTO 2022, Part II (LNCS, Vol. 13508)*, Yevgeniy Dodis and Thomas Shrimpton (Eds.). Springer, Cham, Switzerland, Santa Barbara, CA, USA, 71–101. doi:10.1007/978-3-031-15979-4_3
- [60] Vadim Lyubashevsky, Gregor Seiler, and Patrick Steuer. 2024. The LaZer Library: Lattice-Based Zero Knowledge and Succinct Proofs for Quantum-Safe Privacy. In *ACM CCS 2024*, Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie (Eds.). ACM Press, Salt Lake City, UT, USA, 3125–3137. doi:10.1145/3658644.3690330
- [61] Robert J. McEliece and Dilip V. Sarwate. 1981. On Sharing Secrets and Reed-Solomon Codes. *Commun. ACM* 24, 9 (1981), 583–584. doi:10.1145/358746.358762
- [62] Daniele Micciancio. 2018. On the Hardness of Learning With Errors with Binary Secrets. *Theory of Computing* 14, 13 (2018), 1–17. doi:10.4086/toc.2018.v014a013
- [63] Christian Mouchet, Elliott Bertrand, and Jean-Pierre Hubaux. 2023. An Efficient Threshold Access-Structure for RLWE-Based Multiparty Homomorphic Encryption. *J. Cryptol.* 36, 2 (mar 2023), 20 pages. doi:10.1007/s00145-023-09452-8
- [64] Torben P. Pedersen. 1992. Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing. In *CRYPTO’91 (LNCS, Vol. 576)*, Joan Feigenbaum (Ed.). Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA, 129–140. doi:10.1007/3-540-46766-1_9
- [65] Chris Peikert, Vinod Vaikuntanathan, and Brent Waters. 2008. A Framework for Efficient and Composable Oblivious Transfer. In *CRYPTO 2008 (LNCS, Vol. 5157)*, David Wagner (Ed.). Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA, 554–571. doi:10.1007/978-3-540-85174-5_31
- [66] Rafael Del Pino, Shuichi Katsumata, Guilhem Niot, Michael Reichle, and Kaoru Takemure. 2025. Unmasking Traccoon: A Lattice-Based Threshold Signature with An Efficient Identifiable Abort Protocol. In *CRYPTO 2025, Part VI (LNCS, Vol. 16005)*, Yael Tauman Kalai and Seny F. Kamara (Eds.). Springer, Cham, Switzerland, Santa Barbara, CA, USA, 423–456. doi:10.1007/978-3-032-01887-8_14
- [67] Rafael Del Pino and Guilhem Niot. 2025. Finally! A Compact Lattice-Based Threshold Signature. In *PKC 2025, Part III (LNCS, Vol. 15676)*, Tibor Jager and Jiaxin Pan (Eds.). Springer, Cham, Switzerland, Røros, Norway, 169–199. doi:10.1007/978-3-031-91826-1_6
- [68] Tal Rabin. 1994. Robust Sharing of Secrets When the Dealer is Honest or Cheating. *J. ACM* 41, 6 (1994), 1089–1109. doi:10.1145/195613.195621
- [69] Tal Rabin and Michael Ben-Or. 1989. Verifiable Secret Sharing and Multiparty Protocols with Honest Majority (Extended Abstract). In *21st ACM STOC*. ACM Press, Seattle, WA, USA, 73–85. doi:10.1145/73007.73014
- [70] Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, Damien Stehlé, and Jintai Ding. 2022. *CRYSTALS-KYBER*. Technical Report. National Institute of Standards and Technology. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- [71] Adi Shamir. 1979. How to Share a Secret. *Communications of the Association for Computing Machinery* 22, 11 (Nov. 1979), 612–613. doi:10.1145/359168.359176
- [72] Guofeng Tang, Bo Pang, Long Chen, and Zhenfeng Zhang. 2023. Efficient Lattice-Based Threshold Signatures With Functional Interchangeability. *IEEE Transactions on Information Forensics and Security* 18 (2023), 4173–4187. doi:10.1109/TIFS.2023.3293408
- [73] Stefano Tessaro and Chenzhi Zhu. 2023. Threshold and Multi-signature Schemes from Linear Hash Functions. In *EUROCRYPT 2023, Part V (LNCS, Vol. 14008)*, Carmi Hazy and Martijn Stam (Eds.). Springer, Cham, Switzerland, Lyon, France, 628–658. doi:10.1007/978-3-031-30589-4_22
- [74] Raymond K. Zhao, Ron Steinfeld, and Amin Sakzad. 2020. FACCT: Fast, Compact, and Constant-Time Discrete Gaussian Sampler over Integers. *IEEE Trans. Comput.* 69, 1 (2020), 126–137. doi:10.1109/TC.2019.2940949
- [75] Chenzhi Zhu and Stefano Tessaro. 2025. The Algebraic One-More MISIS Problem and Applications to Threshold Signatures. In *CRYPTO 2025, Part I (LNCS, Vol. 16000)*, Yael Tauman Kalai and Seny F. Kamara (Eds.). Springer, Cham, Switzerland, Santa Barbara, CA, USA, 548–581. doi:10.1007/978-3-032-01855-7_18

A Related Work

A.1 State-of-the-Art

Gur, Katz, and Silde [48] (from now on, GKS) presented the first t -out-of- n threshold signature scheme based on standard lattice assumptions, while avoiding reliance on threshold fully homomorphic encryption (FHE) or generic secure multi-party computation (MPC). They achieved a two-round signing protocol within a framework combining ideas from threshold FHE [14] and n -out-of- n threshold signatures [28], utilizing a non-aborting underlying signature scheme [1]. To do this, they design a linearly homomorphic encryption scheme alongside a novel threshold decryption algorithm, computing the linear operations of the signature scheme in the encrypted domain. However, both signature rounds depend on the message, and the additional use of trapdoor commitments necessitated the inclusion of a commitment opening in the final signature, altering the verification algorithm and effectively doubling the signature size. They also devised a combined distributed key generation protocol for the encryption and signature schemes based on Shamir secret sharing [71], although they do not provide concrete communication costs for this sub-protocol. Their scheme offers identifiable abort using non-interactive zero-knowledge proofs, though full details of how one could instantiate the proofs are not given. They provide a concrete instantiation for only five users which already has very large keys, signatures and communication complexity. Furthermore, they do not provide an implementation of their scheme.

del Pino et al. [30] introduce TRaccoon, a threshold variant of the Raccoon signature scheme [31] which is based on the MLWE-with-hints assumption by Kim et al. [53]. They achieved a three-round scheme with very small public keys, signature sizes, communication costs, and efficient implementation for up to 1024 users. However, the scheme’s drawbacks include the message dependency of all three rounds and the necessity for involved parties to maintain state to prevent secret reuse (due to their masking procedure that safeguards the signing key shares when applying Shamir secret sharing). Their scheme requires a trusted setup (it is not directly compatible with the distributed key generation protocol by [48]) and does not provide identifiable aborts.

Espitau, Katsumata, and Takemure [37] designed a two-round threshold signature scheme founded on a new assumption known as the Algebraic One-More Module Learning With Errors (AOM-MLWE) problem. Their scheme is quite efficient, but the parameters are determined based on their cryptanalysis of this new assumption, proving only selective security of AOM-MLWE to standard lattice problems such as MSIS and MLWE. The first round of their scheme is independent of the message and signing set, and they avoid maintaining a state across different signature procedures. They leave distributed key generation, identifiable aborts, and implementation as future work. A follow up work by Zhu and Tessaro [75] introduced the Algebraic One-More Module Short Integer Solution (AOM-MSIS) assumption, showed a full reduction to standard MSIS and MLWE, as well as a proof for AOM-MLWE against any algebraic adversaries, solving an open problem in [37]. They also provide new parameters based on their security proof, which are slightly worse than the initial instantiation based on the incomplete security reduction. We include both parameter sets.

Katsumata, Reichle, and Takemure [51] present the only lattice-based threshold signature scheme with adaptive security, allowing an adversary to adaptively corrupt users instead of stating up-front which users are corrupted. Their framework is generic, stateless, and works for the Raccoon signature scheme, leading to very efficient public keys and signatures. However, their scheme requires five rounds of communication, of which three are message dependent, making it impractical for real-world usage with many parties. They have not provided communication estimates nor an implementation of their scheme.

Chairattana-Apirom, Tessaro, and Zhu [24] presented a non-interactive two-round threshold signature based on standard lattice assumptions, using Benaloh-Leichter [10] linear secret sharing and linear hash function techniques inspired by Tessaro and Zhu [73]. While their scheme is round-optimal, their public keys, signatures, and communication costs are substantially larger than the other related works. Furthermore, their secret sharing scheme requires each user to (on average) hold secret keys of size 610.7 KB and a distributed key generation protocol would require generic MPC techniques and is not considered practical. The recent work by Zhu and Tessaro [75] improves the construction, leading to slightly better parameters, but the sizes are still larger than other works. Their protocol does not achieve identifiable aborts, however, in contrast to TRaccoon [30] which requires PRFs and uniform masks, this scheme could potentially be extended using lattice-based zero-knowledge proofs similar to the GKS-framework, given that commitments to all of the secret key shares are provided during the trusted setup.

Boschini et al. [17] introduced Ringtail, which is similar in spirit to [37], being a stateless two-round non-interactive scheme, but based on the standard MLWE assumption. Their non-interactive communication is only 10 KB, and they provide an implementation with pre-processing connecting servers across 8 countries in 5 continents showing that the end-to-end latency mostly depends on the network costs, emphasizing the importance of non-interactive schemes. However, Ringtail also requires a trusted setup and does not provide identifiable aborts. Additionally, their security proof and parameter setting assume that the number of hash queries and signatures queries from the adversary are the same, while in practice one wants security against an adversary that can make 2^{128} or even 2^{256} hash queries but can only request 2^{60} signatures. The updated analysis by Zhu and Tessaro [75] provides a security proof based on the AOM-MSIS assumption which avoids this gap, leading to worse parameters.

del Pino and Niot [67] achieved the first practical threshold instantiation of the CRYSTALS-Dilithium signature scheme [36] with small verification key, signature size, and communication. The scheme is a three-round scheme with non-interactive signing. However, they use replicated secret sharing which means that the secret key size is exponential in the number of parties and the scheme does not provide identifiable aborts nor accommodate for a distributed key generation. They provide an implementation for up to 8 parties.

del Pino et al. [66] provides a technique using the lattice-based succinct zero-knowledge proof system LaBRADOR [12] to add identifiable aborts to the Raccoon signature scheme [30], only impacting

the communication of the protocol when signatures fail verification. The extra communication crucially scales linearly with the threshold t .

A.2 Other Related Work

A.2.1 Lattice-Based Secret Sharing. Gentry, Halevi, and Lyubashevsky [45] presented a verifiable secret sharing scheme from lattices for up to 1000 parties. Their construction is based on the lattice-based encryption scheme by Peikert, Vaikuntanathan, and Waters [65] and the Bulletproof zero-knowledge proof system by Bunz et al. [22], however, Bulletproof is based on discrete logarithm assumptions and is not secure against quantum computers. Espitau, Niot, and Prest [38] developed a robust distributed key generation protocol for lattice-signatures, but their scheme requires 2/3 of the parties involved to behave honestly.

A.2.2 Schemes from Generic MPC. Bendlin, Krehbiel, and Peikert [11] used MPC to produce trapdoors for the Gentry, Peikert, Vaikuntanathan signature scheme [46], but this is a theoretical construction that would not be useful in practice. Cozzo and Smart [27] used generic MPC to implement NIST post-quantum signatures, and concluded that “all schemes, except the ones in the MQ family, have significant issues when one wishes to convert them using relatively generic MPC techniques. The lattice based schemes are hampered by requiring a mix of operations which are suited to both linear secret shared-based schemes and garbled circuits-based MPC techniques”. Tang, Pang, and Chen [72] implemented a threshold version of CRYSTALS-Dilithium [58], however, their construction requires 15 rounds of communication for 2-out-of-3 signatures, and scales worse when the number of parties involved increases.

A.2.3 Schemes from Generic FHE. Boneh et al. [14] presented a threshold signature framework based on fully homomorphic encryption, evaluating the complete signature (including rejection sampling and the concrete hash-function used as a random oracle) in the encrypted domain before applying threshold decryption to recover the signature. Agrawal, Stehle, and Yadav [1] improves their construction using rejection-free signatures, however, none of the constructions are practical due to the complex evaluation algorithm, and they did not present a concrete instantiation of the threshold decryption. The latter can be implemented from Mouchet, Bertrand, and Hubaux [63] or using GKS [48].

A.2.4 Full Threshold Schemes and Multisignatures. A line of recent works build full-threshold (n -out-of- n) and multi-signatures from lattice assumptions [3, 18, 25, 28, 41–43], but none of these can easily be adjusted to the arbitrary threshold setting. The t -out-of- n GKS-framework can be seen as combining the full threshold scheme by Damgård et al. [28] with the FHE framework by Boneh et al. [14].

B Additional Background

B.1 Shamir Secret Sharing

The t -out-of- n sharing of $S \in R_q^{m_1 \times m_2}$ proceeds as follows.

- (1) Generate a uniform polynomial $P \in R_q^{m_1 \times m_2}$, conditioned on $P(0) = S$.
- (2) For $i \in [n]$, compute the i th share as $\llbracket S \rrbracket_i := P(i)$.

Given $\mathcal{S} \subseteq [n]$ of size $\geq t$, Lagrange interpolation can be used to reconstruct S as

$$S = \sum_{i \in \mathcal{S}} \lambda_i^{\mathcal{S}} \llbracket S \rrbracket_i, \quad \text{where} \quad \lambda_i^{\mathcal{S}} := \prod_{j \in \mathcal{S} \setminus \{i\}} \frac{j}{j-i}. \quad (3)$$

B.2 Gaussians

In this work we use the standard discrete Gaussian distribution of support $\mathbb{Z}^N$, center 0, and standard deviation σ defined by its probability distribution function

$$\mathcal{D}_\sigma(z) := \frac{\rho_\sigma(z)}{\sum_{z \in \mathbb{Z}^N} \rho_\sigma(z)},$$

where $\rho := \exp\left(-\frac{\|z\|_2^2}{2\sigma^2}\right)$. Moreover, we will always use a *truncated* Gaussian distribution, from which sampling yields N -dimensional integer vectors with bounded infinity and two-norms. Specifically, sampling from this truncated Gaussian will be denoted $'z \leftarrow \mathcal{D}_\sigma'$ ¹⁷ and, for real numbers $t_\infty > 0$ and $t_2 > 1$, is implemented as follows:

Truncated Gaussian Sampling, denoted $'z \leftarrow \mathcal{D}'$

- (1) Sample z from the regular (non-truncated) Gaussian of parameter σ and center 0.
- (2) **if** either $\|z\|_\infty \geq t_\infty \cdot \sigma$ **or** $\|z\|_2 \geq t_2 \cdot \sigma \cdot \sqrt{N}$ **then restart**
- (3) **return** z .

Figure 8: Truncated Gaussian sampling.

We choose t_∞ and t_2 so that Step 2 occurs with small prob..

B.3 Module-SIS/-LWE Hardness Assumptions

Definition 6 (MSIS [55]). *Let k, ℓ be positive integers and $0 < \eta \ll q$. Then, given $A \leftarrow R_q^{k \times \ell}$, the Module-SIS problem asks an adversary $\mathcal{A}$ to find $z \in R_q^\ell$ such that $Az = 0$ and $0 < \|z\|_2 \leq \eta$. $\mathcal{A}$ is said to have advantage ϵ_{MSIS} in solving $\text{MSIS}_{k,\ell,\eta}$ if*

$$\Pr \left[0 < \|z\|_2 \leq \eta \wedge Az = 0 \mid A \leftarrow R_q^{k \times \ell}; z \leftarrow \mathcal{A}(A) \right] \geq \epsilon_{\text{MSIS}}.$$

Definition 7 (MLWE [55]). *Let k, ℓ be positive integers, and χ be a probability distribution over R_q . The Module-LWE problem then asks an adversary $\mathcal{A}$ to distinguish between the following two cases:*

- (1) (A, As) for public $A \leftarrow R_q^{k \times (\ell+k)}$ and secret $s \leftarrow \chi^{\ell+k}$,
- (2) $(A, b) \leftarrow R_q^{k \times (\ell+k)} \times R_q^k$ where both are sampled uniformly.

Then $\mathcal{A}$ is said to have advantage ϵ_{MLWE} in solving $\text{MLWE}_{k,\ell,\chi}$ if

$$\left| \Pr \left[b = 1 \mid A \leftarrow R_q^{k \times (\ell+k)}; s \leftarrow \chi^{\ell+k}; b \leftarrow \mathcal{A}(A, As) \right] - \Pr \left[b = 1 \mid A \leftarrow R_q^{k \times (\ell+k)}; b \leftarrow R_q^k; b \leftarrow \mathcal{A}(A, b) \right] \right| \geq \epsilon_{\text{MLWE}}.$$

¹⁷We abuse this notation in places, for example denoting by $\mathcal{D}_y$ the Gaussian from which secrets s and e are sampled to construct $y := As + e$ as in the signing public key.

B.4 Rènyi Divergence

We rely on the standard notion of Rènyi divergence in this work. Specifically, we require a lemma regarding Rènyi divergence between Gaussians with different centers and the notion of *public sampleability*.

For any two discrete probability distributions ϕ and ϕ' such that $\text{Supp}(\phi) \subseteq \text{Supp}(\phi')$ and an $a \in (1, +\infty)$, the Rènyi divergence of order a can be defined as:

$$R_a(\phi \parallel \phi') := \left(\sum_{x \in \text{Supp}(\phi)} \frac{\phi(x)^a}{\phi'(x)^{a-1}} \right)^{\frac{1}{a-1}}.$$

with the special case $R_\infty(\phi \parallel \phi') := \max_{x \in \text{Supp}(\phi)} \frac{\phi(x)}{\phi'(x)}$. The Rènyi divergence R_a has the following properties for probability distributions ϕ, ϕ', ϕ'' with $\text{Supp}(\phi) \subseteq \text{Supp}(\phi') \subseteq \text{Supp}(\phi'')$:

- **Log. Positivity:** $R_a(\phi \parallel \phi') \geq R_q(\phi \parallel \phi) = 1$.
- **Data Processing Inequality:** $R_a(\phi^f \parallel \phi'^f) \leq R_a(\phi \parallel \phi')$ for any function f where ϕ^f denotes the distribution of $f(y)$ where $y \leftarrow \phi$.
- **Multiplicativity:** Assume ϕ and ϕ' are two distributions for a pair of random variables (Y_0, Y_1) . For $i \in \{0, 1\}$, let ϕ_i denote the marginal distribution of Y_i under ϕ , and let $\phi_{1|0}(\cdot | y_0)$ denote the conditional distribution of Y_1 given $Y_0 = y_0$. Then
 - $R_a(\phi \parallel \phi') = R_a(\phi_0 \parallel \phi'_0) \cdot R_a(\phi_{1|0} \parallel \phi'_{1|0})$ if Y_0 and Y_1 are independent for a in given interval.
 - $R_a(\phi \parallel \phi') \leq R_\infty(\phi_0 \parallel \phi'_0) \cdot \max_{y_0 \in X} R_a(\phi_{1|0}(\cdot | y_0) \parallel \phi'_{1|0}(\cdot | y_0))$
- **Probability Preservation:** Let $E \subseteq \text{Supp}(Q)$ be an arbitrary event. For some a : $\phi'(E) \geq \phi(E)^{\frac{a}{a-1}} / R_a(\phi \parallel \phi')$. Furthermore; $\phi'(E) \geq \phi(E) / R_\infty(\phi \parallel \phi')$.
- **Weak Triangle Equality:** $R_a(\phi \parallel \phi'') \leq R_a(\phi \parallel \phi') \cdot R_\infty(\phi' \parallel \phi'')$ and $R_a(\phi \parallel \phi'') \leq R_\infty(\phi \parallel \phi')^{\frac{a}{a-1}} \cdot R_a(\phi' \parallel \phi'')$.

Lemma 3 ([56, Lemma 4.2]). *Let ϕ and ϕ' be distributions corresponding to Gaussian $\mathcal{D}_{\mathbf{c},s}^m$ and $\mathcal{D}_{\mathbf{c}',s}^m$ with centers $\mathbf{c}$ and $\mathbf{c}'$, and $s \geq \eta(R^m)$. Then for any $a \in (1, +\infty)$:*

$$R_a(\phi \parallel \phi') \leq \exp \left(a \cdot \pi \cdot \frac{\|\mathbf{c} - \mathbf{c}'\|_2^2}{s^2} \right)$$

Lemma 4 ([8, Theorem 4.2]). *Let ϕ, ϕ' denote two distributions with $\text{Supp}(\phi) \subseteq \text{Supp}(\phi')$, and $D_0(r)$ and $D_1(r)$ denote two distributions determined by some parameter $r \in \text{Supp}(\phi')$. Let P, P' be two decision problems defined as follows:*

- *Problem P : Distinguish whether input y is sampled from distribution Y_0 or Y_1 , where*
- $$Y_0 = \{y: r \leftarrow \phi, y \leftarrow D_0(r)\}, \quad Y_1 = \{y: r \leftarrow \phi, y \leftarrow D_1(r)\}$$
- *Problem P' : Distinguish whether input y is sampled from distribution Y'_0 or Y'_1 , where*

$$Y'_0 = \{y: r \leftarrow \phi', y \leftarrow D_0(r)\}, \quad Y'_1 = \{y: r \leftarrow \phi', y \leftarrow D_1(r)\}$$

Assume that $D_0(\cdot)$ and $D_1(\cdot)$ satisfy the following public sampleability property: There exists a sampling algorithm S with runtime T_S such that for all (r, b) , given any sample y from $D_b(r)$ over coins S :

- *$S(0, y)$ outputs a fresh sample distributed as $D_0(r)$.*

- *$S(1, y)$ outputs a fresh sample distributed as $D_1(r)$.*

Then, given a T -time distinguisher $\mathcal{A}$ for problem P with advantage ϵ we can construct a distinguisher $\mathcal{A}'$ for problem P' with runtime and distinguishing advantage respectively bounded from above and below, for any $a \in (1, \infty]$, by:

$$\frac{64}{\epsilon^2} \log \left(\frac{8R_a(\phi \parallel \phi')}{\epsilon^{a/(a-1)+1}} \right) \cdot (T_S + T) \text{ and } \frac{\epsilon}{4 \cdot R_a(\phi \parallel \phi')} \cdot \left(\frac{\epsilon}{2} \right)^{\frac{a}{a-1}}.$$

B.5 Multi-Proof Extractable NIZK

We recall the non-interactive zero-knowledge proof techniques first introduced by Lyubashevsky, Nguyen, and Plançon [59], often referred to as the LNP framework. We will use these techniques to prove knowledge of short $\mathbf{s}_1$ satisfying equations of the form

$$\mathbf{P}\mathbf{s}_1 = \mathbf{v} \text{ over } R_q, \quad (4)$$

where $\mathbf{P} \in R_q^{n \times m}$ and $\mathbf{v} \in R_q^m$ are public matrices and vectors (resp.) of elements in R_q . In particular, we recall the multi-proof extractable¹⁸ variant which is obtained after applying the Katsumata transform [50] to the LNP framework [59], as shown in [15]. We require this variant in order to prove the unforgeability property of our signature scheme wherein efficient extractors cannot be constructed from traditional rewinding arguments.

Our relations fit into this framework natively since parts of the relations to be proven already include the “additional” information required as part of the transform. Informally, Katsumata transform needs additional extractable commitments to the witness as part of the proof so the extractor can extract without rewinding. Since we are using BDLOP commitments as shown in Section 2.4, all we need to have is commitments to our witnesses. In Appendix D we will see that each relation contains commitments to the witnesses explicitly, which help us achieve straight-line extractability using multi-proof extractable NIZKs and BDLOP commitments without any additional cost.

B.6 Threshold Homomorphic Encryption

A homomorphic encryption scheme $\mathcal{E}$, given a set of possible circuits $\mathcal{F}$, consists of the following algorithms:

- $\text{KGen}_{\mathcal{E}}$ is a probabilistic algorithm that outputs a public encryption key $\text{pk}_{\mathcal{E}}$ and a decryption key $\text{sk}_{\mathcal{E}}$.
- Enc is a probabilistic algorithm that takes as input a public key $\text{pk}_{\mathcal{E}}$ and a plaintext ptx , and outputs a ciphertext ctx .
- Eval is a deterministic algorithm that takes as input a circuit $F \in \mathcal{F}$ and a list of ciphertexts $\text{ctx}_1, \dots, \text{ctx}_k$, and outputs a ciphertext ctx^* .
- Dec is a deterministic algorithm that takes as input a decryption key $\text{sk}_{\mathcal{E}}$ and a ciphertext ctx^* , and outputs a plaintext ptx .

Notationally, we allow algorithms to take as inputs a list of plaintexts, ciphertexts, or keys to denote that they are applied on each input individually. Correctness and ciphertext indistinguishability are standard, and thus we omit the definitions here.

We now extend the above to accommodate $\mathcal{E}$ with distributed key generation and (non-interactive) threshold decryption:

¹⁸This is sometimes called ‘online’ or ‘straight-line’ extractable.

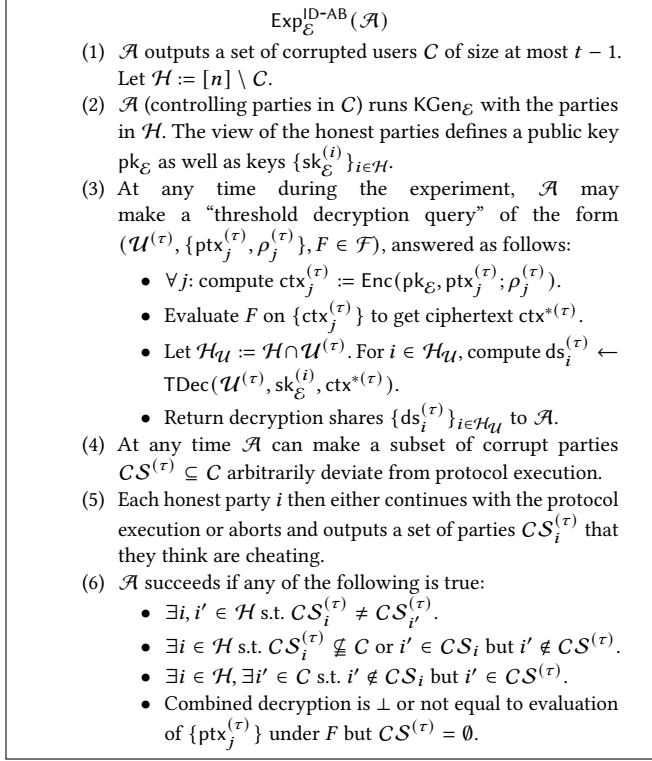


Figure 10: Experiment $\text{Exp}_{\mathcal{E}}^{\text{ID-AB}}(\mathcal{A})$ parameterized by (t, n) .

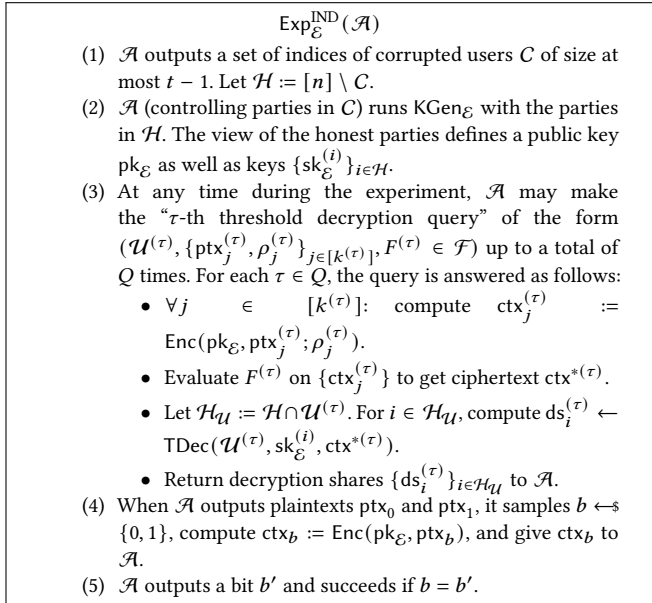


Figure 9: Experiment $\text{Exp}_{\mathcal{E}}^{\text{IND}}(\mathcal{A})$ parameterized by (t, n) .

- $\text{KGen}_{\mathcal{E}}$ is an interactive protocol run by n users, on common input a threshold t . Either all honest users output $\perp$, or they each output a decryption key share $\text{sk}_{\mathcal{E}}^{(i)}$ and a (common) public key $\text{pk}_{\mathcal{E}}$.

- TDec is a probabilistic algorithm that takes as input a set $\mathcal{U}$ consisting of t users, a decryption key share $\text{sk}_{\mathcal{E}}^{(i)}$, and a ciphertext ctx^* , and outputs a partial decryption share ds_i .
- Comb is a deterministic algorithm that takes as input a set $\mathcal{U}$ consisting of t users, a ciphertext ctx^* , and a set of decryption shares $\{\text{ds}_i\}_{i \in \mathcal{U}}$, and outputs a plaintext ptx .

We define security in the threshold setting as follows:

Indistinguishability. $\mathcal{E}$ is (T, ϵ) -threshold indistinguishable if the probability of any adversary $\mathcal{A}$ running in time T succeeding in experiment $\text{Exp}_{\mathcal{E}}^{\text{IND}}(\mathcal{A})$ as depicted in Figure 9 is at most $1/2 + \epsilon$.

Similar to threshold signatures, we also define security with identifiable aborts for threshold homomorphic encryption.

Security with Identifiable Aborts $\mathcal{E}$ has (T, ϵ) -security with identifiable aborts if the probability of any adversary $\mathcal{A}$ running in time T succeeding in experiment $\text{Exp}_{\mathcal{E}}^{\text{ID-AB}}(\mathcal{A})$ as depicted in Figure 10 is at most ϵ .

Additionally, we use the standard notion of commitments with *hiding* and *binding* properties. We instantiate our commitments using BDLOP commitment scheme defined in Section 2.4.

C Omitted Proofs

C.1 VSS Proofs

Lemma 5 (Privacy). *The WSS presented above is private.*

PROOF. Let C denote the set of indices of parties controlled by the (static) adversary. We show that, if the dealer is honest then, at the end of the sharing phase, the view of the adversary is independent of the dealer’s input S . In the real world, the view and output of the adversary are

$$\{\{\text{com}[\![S]\!]_i\}_{i \in [n]}, \pi_{\text{VSS}}\} \quad \text{and} \quad \{\{\![S]\!]_i, r_{\![S]\!}_i\}_{i \in C},$$

respectively. By the zero-knowledge property of π_{VSS} , and the hiding property of the BDLOP commitment, all proofs and commitments with $i \in [n] \setminus C$ can be simulated. Furthermore, since the sharing polynomials have degree $< t$ and $|C| < t$, a standard argument implies $\{\{\![S]\!]_i, r_{\![S]\!}_i\}_{i \in C}$ reveals nothing about S . $\square$

Lemma 6 (Correctness). *The WSS presented above is correct.*

PROOF. Assuming an honest dealer, the correctness of π_{VSS} and of the BDLOP commitment scheme ensure that each party outputs some $S_i \neq \perp$ after the sharing phase. We aim to show that $S_i = \![S]\!]_i$.

Owing to the correctness of the constituent proofs π_1, π_2 , and π_3 , we have that the committed S is short, that the inner product of a random dual code word with the set of shares S_i is zero, and that a subset of size t of the S_i reconstructs to S . By Lemma 2 have that that with overwhelming probability, the S_i do are correct sharings of S and by the correctness of the BDLOP, we have $S_i = \![S]\!]_i$. $\square$

Lemma 7 (Weak Commitment). *Our WSS satisfies the weak commitment property.*

PROOF. Observe that if verification of π_{VSS} fails, all honest parties will output $\perp$ (i.e. the view of honest parties defines the value

$\perp$). Similarly, all honest parties will output $\perp$ in the reconstruction phase.

We now proceed assuming no party aborts in the sharing phase. This scenario only arises if π_{VSS} verifies correctly. Then by the knowledge soundness of π_{VSS} , there exists a polynomial time extractor Ext (run by the simulator) which extracts a witness to $\mathcal{R}_{VSS}^*$ with overwhelming probability. Such a witness defines a value $S^* \in R_q^{m_1 \times m_2}$ (the object that has been shared).

Now suppose a set $\mathcal{S}$ of $\geq t$ parties run the reconstruction phase. Without loss of generality assume $|\mathcal{S}| = t$ (e.g. by selecting a subset of size t is necessary). If, for honest $\mathcal{P}_i \in \mathcal{S}$, the check $\text{Open}(\text{pk}_{\text{Com}}, \text{com}[\llbracket S \rrbracket_j, d = (\llbracket S \rrbracket_j, r[\llbracket S \rrbracket_j, 1)) = 1$ fails for any $j \in \mathcal{S} \setminus \{i\}$, then $\mathcal{P}_i$ outputs $\perp$. Since this check is performed by all $\mathcal{P}_i \in \mathcal{S}$, if one honest party outputs $\perp$, all honest parties output $\perp$. Assuming that no honest party outputs $\perp$, the soundness of π_{VSS} ensures that all S_i values broadcast by members of $\mathcal{S}$ in reconstruction are genuine shares. By a similar argument to that used to prove correctness of our WSS, conditions 1 and 2 of Definition 5 are satisfied. In particular, all honest parties in $\mathcal{S}$ output S^* . $\square$

C.2 Proof of Theorem 3

PROOF. We prove security through a series of experiments.

Experiment G_0 . The first experiment corresponds to the threshold ciphertext indistinguishability experiment. $\mathcal{A}$ first outputs a set of corrupted users $\mathcal{C}$ of size at most $t - 1$ views of which are always given to $\mathcal{A}$ throughout the game. The challenger then runs $\text{KGen}_{\mathcal{E}}$ to generate public keys and the secret key shares sk_i and sends keys for corrupted parties to $\mathcal{A}$. $\mathcal{A}$ then makes threshold decryption queries for different sets of users $\mathcal{U}$ and messages $\mathbf{m}$. Challenger encrypts $\mathbf{m}$ and computes partial shares for each participant in $\mathcal{U}$ and gives them to $\mathcal{A}$. At the end, $\mathcal{A}$ outputs two messages $\mathbf{m}_0$ and $\mathbf{m}_1$. The challenger randomly chooses one of the messages and returns the ciphertext to $\mathcal{A}$. If $\mathcal{A}$ guesses correctly, it is said to win the game.

$$\Pr[G_0] \leq \text{Adv}_{\mathcal{E}}^{\text{IND}}(\mathcal{A}).$$

Experiment G_1 . We first change how $\pi_{\text{KGen}_{\mathcal{E}}}$ is computed for all honest parties. Instead of honestly computing $\pi_{\text{KGen}_{\mathcal{E}}}$ each honest party calls the simulator $\text{Sim}_{\text{KGen}_{\mathcal{E}}}$ and send a simulated proof instead. The rest of the experiment is the same. Only difference between G_0 , G_1 is then how NIZKAoKs are computed for honest parties. By the zero-knowledge property of the underlying proof system, these are indistinguishable. We then have:

$$|\Pr[G_1] - \Pr[G_0]| \leq |\mathcal{H}| \cdot \epsilon_{\text{ZK}}.$$

Experiment G_2 . Now, we extract $\{\text{sk}_{j,k}\}_{k \in [n]}$ for each corrupted party $\mathcal{P}_j$. When honest parties receive $\pi_{\text{KGen}_{\mathcal{E}}}$ from the corrupted parties, they call $\text{Extract}_{\text{KGen}_{\mathcal{E}}}$ to obtain $mS_{j,k}, \rho_{j,k}$ for all $j \in \mathcal{C}, k \in [n]$, abort if the extraction fails. Then honest parties continue to compute sk_j . The rest is the same. G_2 is then indistinguishable from G_1 as long as the extraction from $\pi_{\text{KGen}_{\mathcal{E}}}$ does not fail. Hence:

$$|\Pr[G_2] - \Pr[G_1]| \leq |\mathcal{C}| \cdot \epsilon_{\text{extract}}.$$

Experiment G_3 . We replace how partial decryptions are computed for one of the honest parties. Let $\mathbf{m}$ be the message in a decryption query, and let $(\mathbf{u}, \mathbf{v})$ be the (honestly generated) encryption of $\mathbf{m}$. Fix an index $i \in \mathcal{U}$ and compute ds_j for $j \neq i \in \mathcal{U}$ as in the

previous experiment. Then set the i th partial decryption share equal to $\text{ds}'_i := \mathbf{v} - \mathbf{m} - \sum_{j \in \mathcal{U}, j \neq i} \lambda_j S_j^\top \mathbf{u} + qE'_i \pmod{\hat{q}}$. The rest of the experiment is the same.

In G_2 the partial decryption share ds_i is computed as $\text{ds}_i := \lambda_i S_i^\top \mathbf{u} + qE_i$, whereas here

$$\begin{aligned} \text{ds}'_i &= \mathbf{v} - \mathbf{m} - \sum_{j \in \mathcal{U}, j \neq i} \lambda_j S_j^\top \mathbf{u} + qE'_i \\ &= \mathbf{v} - \mathbf{m} - (S_{\mathcal{E}}^\top \mathbf{u} - \lambda_i S_i^\top \mathbf{u}) + qE'_i \\ &= \mathbf{v} - S_{\mathcal{E}}^\top \mathbf{u} - \mathbf{m} + \lambda_i S_i^\top \mathbf{u} + qE'_i. \end{aligned}$$

For Q partial decryption queries and correct choice of E_i , we want to show ds_i and ds'_i indistinguishable using R nyi divergence. To invoke Theorem 4, we have to show the public sampleability property is satisfied. Define the randomness r as the additional terms in each of i 's decryption shares $\text{ds}_i = \mathbf{v} - \mathbf{m} - \sum_{j \in \mathcal{U}, j \neq i} \lambda_j S_j^\top \mathbf{u} + r$. In G_0 and G_1 we have $r = -(\mathbf{v} - S_{\mathcal{E}}^\top \mathbf{u} - \mathbf{m}) + qE_i$ and $r = qE_i$ respectively. Then consider $\mathcal{A}$'s view in the indistinguishability game as the following distribution:

$$D_b(r) = \{\text{pk}_{\mathcal{E}}, \{S_j\}_{j \in \mathcal{C}}, \{\text{ptx}^{(\tau)}, \rho^{(\tau)}, \mathcal{U}^{(\tau)}, \{\text{ds}_j^{(\tau)}\}_{j \in \mathcal{U}, \tau \in [Q]}, \text{ptx}_0, \text{ptx}_1, \text{ctx}_b\}$$

To show $D_b(r)$ is publicly sampleable, we define the sampler $S(b', y)$ that on input $b' \in \{0, 1\}$ and a sample y from distribution $D_b(r)$, we can generate a sample from $D_{b'}(r)$ using y . $S(b', y)$ is defined as follows: Upon receiving

$$y = \{\text{pk}_{\mathcal{E}}, \{S_j\}_{j \in \mathcal{C}}, \{\text{ptx}^{(\tau)}, \rho^{(\tau)}, \mathcal{U}^{(\tau)}, \{\text{ds}_j^{(\tau)}\}_{j \in \mathcal{U}, \tau \in [Q]}, \text{ptx}_0, \text{ptx}_1, \text{ctx}_b\},$$

replace $\{\{\text{ds}_j^{(\tau)}\}_{j \in \mathcal{U}, \tau \in [Q]}\}$ with $\text{ds}_j^{(\tau)} = \mathbf{v} - \mathbf{m} - \sum_{j \in \mathcal{U}, j \neq i} \lambda_j S_j^\top \mathbf{u} + r_{b'}$. Computing $\mathbf{v} - \mathbf{m} - \sum_{j \in \mathcal{U}, j \neq i} \lambda_j S_j^\top \mathbf{u}$ requires the knowledge of $\mathbf{v} - S_{\mathcal{E}}^\top \mathbf{u} - \mathbf{m}$, which can be done publicly since randomness for the ciphertext $\rho^{(\tau)}$ is given as part of y . Thus we conclude that $D_b(r)$ is publicly sampleable. We can now invoke Lemma 3 on r for E_i sampled from a Gaussian with parameter σ with centers $(\mathbf{v} - S_{\mathcal{E}}^\top \mathbf{u} - \mathbf{m})$ and 0 for $D_0(r)$ and $D_1(r)$ respectively. The R nyi divergence of rank a between distributions $D_0(r)$ and $D_1(r)$ at (τ) th decryption query is then defined as:

$$R_a(D_0^{(\tau)}(r) || D_1^{(\tau)}(r)) \leq \exp \left(a \cdot \pi \cdot \frac{\|\mathbf{v} - S_{\mathcal{E}}^\top \mathbf{u} - \mathbf{m}\|_2^2}{q^2 \sigma^2} \right)$$

Using the multiplicative property for aggregating Q queries, we have:

$$R_a(D_0(r) || D_1(r)) \leq \exp \left(a \cdot \pi \cdot Q \cdot \frac{\|\mathbf{v} - S_{\mathcal{E}}^\top \mathbf{u} - \mathbf{m}\|_2^2}{q^2 \sigma^2} \right)$$

Since $(\mathbf{u}, \mathbf{v})$ is correctly generated we have:

$$\begin{aligned} \mathbf{v} - S_{\mathcal{E}}^\top \mathbf{u} - \mathbf{m} &= B_{\mathcal{E}}^\top \mathbf{r} + q \cdot \mathbf{e}_2 + \mathbf{m} - S_{\mathcal{E}}^\top (A_{\mathcal{E}}^\top \mathbf{r} + q \cdot \mathbf{e}_1) - \mathbf{m} \\ &= S_{\mathcal{E}}^\top A_{\mathcal{E}}^\top \mathbf{r} + qE_{\mathcal{E}}^\top \mathbf{r} + q \cdot \mathbf{e}_2 - S_{\mathcal{E}}^\top A_{\mathcal{E}}^\top \mathbf{r} - qS_{\mathcal{E}}^\top \cdot \mathbf{e}_1 \\ &= q(E_{\mathcal{E}}^\top \mathbf{r} + \mathbf{e}_2 - S_{\mathcal{E}}^\top \cdot \mathbf{e}_1) \end{aligned}$$

which is the noise in the ciphertext. Hence

$$\leq \exp \left(a \cdot \pi \cdot Q \cdot \frac{B_2^2}{q^2 \sigma_{\text{TDec}}^2} \right).$$

If $\sigma_{\text{Dec}} \geq \sqrt{\frac{Q \cdot B_2^2}{q^2}}$, we have $R_a(D_0(r) || R_a(D_1(r))) \leq \exp(\pi \cdot a)$. Using Theorem 4 we have:

$$\Pr[\mathbf{G}_3] = \frac{\Pr[\mathbf{G}_2]}{\exp(\pi \cdot a)} \cdot \left(\frac{\Pr[\mathbf{G}_0]}{2} \right)^{\frac{a}{a-1}}$$

Experiment G_4 . We replace the rest of the honest shares, i.e., all but the special party i . Instead of honestly computing ds_j for $j \in \mathcal{H}_U$, $j \neq i$, the simulator samples a uniform $ds_j \in R_q$. In addition, ds'_i is now replaced with $ds''_i := \mathbf{v} - \mathbf{m} - \sum_{j \in \mathcal{C}_U} \lambda_j S_j^\top \mathbf{u} - \sum_{j \in \mathcal{H}_U, j \neq i} ds_j \mod q + qE'_i \mod \hat{q}$. The rest is the same as G_3 . We need to show both ds''_i and $\{ds_j\}_{j \in \mathcal{H}_U, j \neq i}$ are indistinguishable from their counterparts in G_3 . We first show the former assuming the latter. In G_3 , ds'_i is computed as $\mathbf{v} - \mathbf{m} - \sum_{j \in \mathcal{U}, j \neq i} \lambda_j S_j^\top \mathbf{u} + qE'_i$ whereas $ds''_i = \mathbf{v} - \mathbf{m} - \sum_{j \in \mathcal{C}_U} \lambda_j S_j^\top \mathbf{u} - \sum_{j \in \mathcal{H}_U, j \neq i} ds_j \mod q + qE'_i \mod \hat{q}$. If $\{ds_j\}_{j \in \mathcal{H}_U, j \neq i}$ sampled from uniform is indistinguishable from $\{ds_j\}_{j \in \mathcal{H}_U, j \neq i}$ computed as $ds_j = \lambda_j S_j^\top \mathbf{u} + qE_j$, then the distributions of $\sum_{j \in \mathcal{H}_U, j \neq i} ds_j \mod q$ are indistinguishable from $\sum_{j \in \mathcal{H}_U, j \neq i} \lambda_j S_j^\top \mathbf{u}$ since $ds_j \mod q = \lambda_j S_j^\top \mathbf{u} + qE_j \mod q = \lambda_j S_j^\top \mathbf{u}$. Then if $\sum_{j \in \mathcal{H}_U, j \neq i} ds_j \mod q$ and $\sum_{j \in \mathcal{H}_U, j \neq i} \lambda_j S_j^\top \mathbf{u}$ are indistinguishable, so are ds'_i and ds''_i .

All it remains to show that $\lambda_j S_j^\top \mathbf{u} + qE_j$ is indistinguishable from uniform. Since λ_j are invertible in $\mathbb{Z}_{\hat{q}}$, if we can argue $\lambda \mathbf{AS} + q\mathbf{E} \approx \mathbf{AS} + \lambda^{-1}q\mathbf{E} = \mathbf{AS} + q'\mathbf{E} \approx \mathbf{AS} + \mathbf{E}$ for uniform $\mathbf{A}$ and $\mathbf{S}$, invertible q' , and relatively short $\mathbf{E}$ then we can argue $\lambda \mathbf{AS} + q\mathbf{E}$ is indistinguishable from uniform.

We first argue that $\mathcal{A}$ distinguishing $S_j^\top \mathbf{u} + \lambda_j^{-1}qE_j$ from uniform ds_j can be used to break MLWE: The MLWE distinguisher interacts with $\mathcal{A}$ and uses its replies in G_2 to answer MLWE challenges. The distinguisher obtains an MLWE challenge $(\mathbf{A}, \mathbf{u})$, for each partial decryption query and sets partial ciphertext $\mathbf{u} = \mathbf{A}_i$ and partial decryption share $ds_j = \mathbf{u}_i$. During each query, if $\mathcal{A}$ behaves noticeably different than G_1 , the MLWE distinguisher replies that $\mathbf{u}_i$ is uniform.

Since λ_j is invertible and q is a prime, λ_j^{-1} exists and $\lambda^{-1}q$ is relatively prime with $\hat{q}$. Hence, if $\mathbf{u}$ is an MLWE instance, then there is no reason for the adversary to behave different than G_1 since $\mathbf{A}_j S_j + \mathbf{E}_j$ is indistinguishable from $\mathbf{A}_j S_j + \lambda_j^{-1}qE_j$ for $\lambda_j^{-1}q$ relatively prime with $\hat{q}$ [20]. If $\mathbf{u}_i$ is uniform, any significant advantage $\mathcal{A}$ has is MLWE distinguisher advantage. Hence we conclude $S_j^\top \mathbf{u} + \lambda_j^{-1}qE_j$ is indistinguishable from uniform ds_j .

If $S_j^\top \mathbf{u} + \lambda_j^{-1}qE_j$ is indistinguishable from uniform ds_j then $\lambda_j(S_j^\top \mathbf{u} + \lambda_j^{-1}qE_j) = \lambda_j S_j^\top \mathbf{u} + qE_j$ is indistinguishable from $\lambda_j ds_j$. Since ds_j is uniform in R_q , so is $\lambda_j ds_j$. Thus we conclude that $\lambda_j S_j^\top \mathbf{u} + qE_j$ is indistinguishable from uniform ds_j by MLWE $_{N,q,\mathcal{D}_{\text{MLWE}}}$ assumption.

$$|\Pr[\mathbf{G}_4] - \Pr[\mathbf{G}_3]| \leq (|\mathcal{H}_U| - 1) \cdot \epsilon_{\text{MLWE}}.$$

Experiment G_5 . Now that partial decryption does not rely on combined secret key, for i we now replace the public key share $\mathbf{B}_i$ for honest parties. Sample a random $\mathbf{B}_i$ and t -out-of- n secret share it into $\{\mathbf{B}_{i,j}\}_{j \in [n]}$. Then derive $\mathbf{S}_{i,j}, \mathbf{E}_{i,j}$ from $\mathbf{B}_{i,j}$. The rest of the experiment remains the same.

We again show that $\mathcal{A}$ distinguishing between G_4 and G_5 can be used to break MLWE assumption. We initiate the challenger for MLWE $_{N,q,\mathcal{D}_{\text{MLWE}}}$ with $\mathbf{A}$ being the uniform element. We then

forward the answer of the challenger as $\mathbf{B}_i$. If the $\mathcal{A}$ behaves significantly differently than G_2 we reply the challenger as the sample being uniform.

$\mathbf{B}_i = \mathbf{AS}_i + q\mathbf{E}_i$ is an MLWE sample hence if the challenger returns an MLWE sample, $\mathcal{A}$ has no reason to behave differently. If the challenger returns a uniform sample, however, any non-negligible advantage $\mathcal{A}$ has is non-negligible advantage in answering challenger's queries since any behavioral difference is forwarded directly. Since $\mathbf{B}_i$ is t -out-of- n secret shared, each $\mathbf{B}_{i,j}$ is uniform in R_q . This is statistically indistinguishable from the real execution where $\mathbf{S}_j$ and $\mathbf{E}_j$ are t -out-of- n shared therefore $\mathbf{S}_{i,j}, \mathbf{E}_{i,j}$ and consequently $\mathbf{B}_i = \mathbf{AS}_i + q\mathbf{E}_i$ is uniform in R_q . G_3 is then indistinguishable from G_2 by MLWE assumption and $\mathcal{A}$'s distinguishing advantage can be bounded by ϵ_{MLWE} .

$$|\Pr[\mathbf{G}_5] - \Pr[\mathbf{G}_4]| \leq |\mathcal{H}| \cdot \epsilon_{\text{MLWE}}.$$

Experiment G_6 . Since public key shares are also independent, we now replace unopened commitments with random elements. Instead of committing to $\{\mathbf{S}_{i,j}\}$, honest parties commit to random matrices instead. By the hiding property of Com,

$$|\Pr[\mathbf{G}_6] - \Pr[\mathbf{G}_5]| \leq |\mathcal{H}| \cdot \epsilon_{\text{MLWE}}.$$

Experiment G_7 . We now derive $\mathbf{B}_i$ a posteriori. Before step 1 of KGen $_{\mathcal{E}}$, $\mathcal{B}$ receives a BGV key pair $(\mathbf{A}, \mathbf{B}_{\mathcal{E}})$. Instead of honestly computing $\mathbf{H}$, $\mathcal{P}_i$ samples a random $h_i \leftarrow \{0, 1\}^\lambda$ instead. Then derive $\mathbf{B}_i := \mathbf{B}_{\mathcal{E}} - \sum_{j \neq i \in [n]} \mathbf{B}_j$ and programs $\mathbf{H}$ such that $h_i = \mathbf{H}(i, \mathbf{B}_i)$.

The rest of the experiment is the same as before.

By the MLWE assumption, $\mathbf{B}_{\mathcal{E}}$ is indistinguishable from uniform. Then as long as programming of $\mathbf{H}$ does not fail, G_6 and G_7 are indistinguishable.

$$|\Pr[\mathbf{G}_7] - \Pr[\mathbf{G}_6]| = \frac{QH}{2^\lambda}.$$

We now show that if $\mathcal{A}$ wins G_7 , then $\mathcal{A}$ can be used to break the IND-CPA security of BGV scheme. $\mathcal{B}$ interacts with the challenger to IND-CPA security of BGV and obtains $\mathbf{A}, \mathbf{B}$ which then are used during KGen $_{\mathcal{E}}$. Whenever $\mathcal{A}$ sends the challenge plaintexts $\mathbf{m}_0$ and $\mathbf{m}_1$ $\mathcal{B}$ forwards them to the BGV challenger and obtains the ciphertext ctx^* . $\mathcal{B}$ sends ctx^* along with its simulated proof to $\mathcal{A}$. When $\mathcal{A}$ replies with 0 or 1, $\mathcal{B}$ forwards the response to BGV challenger.

The public key $\mathcal{A}$ generates as part of KGen $_{\mathcal{E}}$ contains the same $(\mathbf{A}, \mathbf{B})$ as $\mathcal{B}$ received from the BGV oracle as a challenge. Hence ctx^* received by $\mathcal{B}$ and consequently $\mathcal{A}$ is a BGV ciphertext encrypted under $\mathbf{A}, \mathbf{B}$ which is indistinguishable from ctx derived as part of the protocol execution. Then, a correct answer given by $\mathcal{A}$ is also a correct answer for $\mathcal{B}$ for the encryption oracle.

$$|\Pr[\mathbf{G}_7]| = \epsilon_{\text{IND-CPA}}.$$

This concludes the proof. $\square$

C.3 Proof of Theorem 4

PROOF. We first start by observing the fact in the synchronous model with access to an authenticated broadcast, each honest party has access to the same communication transcript. Then, throughout the protocol every check and consequent abort by honest parties will have the same result therefore will be consistent. Hence any

corrupted set CS output by honest parties will be the same and will not include any honest parties. Similarly, if any corrupted user does not send their message, it will be noticed by all honest parties looking at the transcript. Then by assumption $\mathcal{A}$ cannot succeed in $\text{Exp}_S^{\text{ID-AB}}$ by first two conditions.

If $\mathcal{A}$ does not cause abort, then protocol proceeds as intended and by the correctness of the scheme there exists no $\{\text{ptx}_j\}$ or $F \in \mathcal{F}$ such that decryption fails. Thus, $\mathcal{A}$ cannot win through fourth condition either.

All it remains to argue is $\mathcal{A}$ cannot win through the third condition which we now show cannot be the case for any of the checks in protocol.

During $\text{KGen}_{\mathcal{E}}$, $\mathcal{A}$ can only succeed if any $j \in C$ can send h_j , $\pi_{\text{KGen}_{\mathcal{E}}}^{(j)}$, $\{\text{com}_{j,k}\}$ that is inconsistent with the protocol information an not detected by any user in $\mathcal{H}$. This can only be the case if (i) j found $\mathbf{B}_j^* \neq \mathbf{B}_j$ where $H(j, \mathbf{B}_j^*) = H(j, \mathbf{B}_j)$ (ii) j found $\mathbf{S}_{j,k}^*, \rho_{j,k}^* \neq \mathbf{S}_{j,k}, \rho_{j,k}$ where $\text{com}_{j,k} = \text{Com}(\mathbf{S}_{j,k}^*, \rho_{j,k}^*)$ or (iii) j sent $\pi_{\text{KGen}_{\mathcal{E}}}^{(j)}$ without knowledge of satisfying $\mathbf{S}_j, \mathbf{E}_j$ or their t -out-of- n shares. If (i) is the case, then j found a collision for H , which does not happen for all but negligible probability. If (ii) is the case, then j can be used as a subroutine to break the binding property of the commitment of the underlying commitment scheme Com . If (iii) is the case however j can be used as a cheating prover for the underlying NIZKAoK $\Pi_{\text{KGen}_{\mathcal{E}}}$, which once again cannot happen for all but negligible probability by the knowledge soundness assumption.

Since F in question is public, final ctx^* will be consistent for each honest party. For TDec and Comb , $\mathcal{A}$ can only succeed if j can send a ds_j without correctly computing it. If that's the case, j used as a cheating prover for the underlying NIZKAoK Π_{ds} , which once again cannot happen for all but negligible probability by the knowledge soundness assumption.

Since $\mathcal{A}$ cannot make any $j \in C$ cheat without detection with non-negligible probability, it cannot succeed in the identifiable aborts game. This concludes the proof. $\square$

C.4 Proof of Theorem 6

PROOF. It is clear that the check $c = c^*$ holds. Assuming the correctness of the linearly homomorphic threshold encryption scheme in Figure 5, we have that

$$\mathbf{z} = c \cdot \mathbf{s} + \sum_{j \in \mathcal{U}} \mathbf{r}_j.$$

Then, by an analogous argument to [48, Lemma C.1], the verification check $\|\mathbf{z}, 2^{\kappa_w} \cdot \mathbf{h}\| \leq B_{2,z}$ holds if

$$B_{2,z} = e^{1/4} \cdot (n\nu\sigma_y + t\sigma_w)\sqrt{N(k+\ell)} + (\nu \cdot 2^{\kappa_y} + 2^{\kappa_w+1}) \cdot \sqrt{Nk}.^{19}$$

It now remains to show the correctness of the linearly homomorphic encryption scheme. More precisely, we must choose parameters so that

$$\text{Comb}(\text{ctx}_z, \{\text{ds}_j\}_{j \in \mathcal{U}}) = c \cdot \mathbf{s} + \sum_{j \in \mathcal{U}} \mathbf{r}_j,$$

¹⁹Note that while $\mathbf{s}_i$ and $\mathbf{r}_i$ are sampled as Gaussians, the sum is not guaranteed to be a Gaussian since we only check for bounds in our NIZKAoKs.

with probability 1. Denoting by $\text{ctx}_z = (\mathbf{v}_z, \mathbf{u}_z)$ the encryption of $\mathbf{z}$, we have that

$$\text{Comb}(\text{ctx}_z, \{\text{ds}_j\}_{j \in \mathcal{U}}) = (\mathbf{v}_z - \sum_{j \in \mathcal{U}} \text{ds}_{z,j}) \bmod \hat{q} \bmod q,$$

where $\text{ds}_{z,j}$ is the decryption share of user j performing partial decryption of ctx_z . Thus, for correct combination, it suffices to set parameters such that

$$\|\mathbf{v}_z - \sum_{j \in \mathcal{U}} \text{ds}_{z,j}\|_{\infty} < \hat{q}/2.$$

Now consider

$$\begin{aligned} \mathbf{v}_z - \sum_{j \in \mathcal{U}} \text{ds}_{z,j} &= (c \cdot \sum_{i \in [n]} \mathbf{v}_{\mathbf{s}_i} + \sum_{j \in \mathcal{U}} \mathbf{v}_{\mathbf{r}_j}) - \mathbf{S}_{\mathcal{E}}^{\top} (c \cdot \sum_{i \in [n]} \mathbf{u}_{\mathbf{s}_i} + \sum_{j \in \mathcal{U}} \mathbf{u}_{\mathbf{r}_j}) \\ &\quad - q \cdot \sum_{j \in \mathcal{U}} E_j, \end{aligned}$$

where $\mathbf{S}_{\mathcal{E}} := \sum_{j \in \mathcal{U}} \lambda_j \mathbf{S}_{\mathcal{E}}^{(j)}$ and λ_j is the j 'th Lagrange coefficient. Then,

$$\begin{aligned} \mathbf{v}_z - \sum_{j \in \mathcal{U}} \text{ds}_{z,j} &= c \cdot \sum_{i \in [n]} (\mathbf{B}_{\mathcal{E}}^{\top} \mathbf{r}_{\mathbf{s}_i} + q \cdot \mathbf{e}_{\mathbf{s}_i,2} + \mathbf{s}_i) \\ &\quad - c \cdot \sum_{i \in [n]} \mathbf{S}_{\mathcal{E}}^{\top} (\mathbf{A}_{\mathcal{E}}^{\top} \mathbf{r}_{\mathbf{s}_i} + q \cdot \mathbf{e}_{\mathbf{s}_i,1}) \\ &\quad + \sum_{j \in \mathcal{U}} (\mathbf{v}_{\mathbf{r}_j} - \mathbf{S}_{\mathcal{E}}^{\top} \mathbf{u}_{\mathbf{r}_j}) - q \cdot \sum_{j \in \mathcal{U}} E_j \\ &= (c \cdot \sum_{i \in [n]} \mathbf{s}_i + \sum_{j \in \mathcal{U}} \mathbf{r}_j) \\ &\quad + c \cdot \sum_{i \in [n]} (\mathbf{E}_{\mathcal{E}}^{\top} \mathbf{r}_{\mathbf{s}_i} + q \cdot \mathbf{e}_{\mathbf{s}_i,2} - q \cdot \mathbf{S}_{\mathcal{E}}^{\top} \mathbf{e}_{\mathbf{s}_i,1}) \\ &\quad + \sum_{j \in \mathcal{U}} (\mathbf{E}_{\mathcal{E}}^{\top} \mathbf{r}_{\mathbf{r}_j} + q \cdot \mathbf{e}_{\mathbf{r}_j,2} - q \cdot \mathbf{S}_{\mathcal{E}}^{\top} \mathbf{e}_{\mathbf{r}_j,1}) - q \cdot \sum_{j \in \mathcal{U}} E_j. \end{aligned}$$

Thus

$$\begin{aligned} \|\mathbf{v}_z - \sum_{j \in \mathcal{U}} \text{ds}_{z,j}\|_{\infty} &\leq \nu \cdot n \cdot t_2 \cdot \sigma_y \cdot \sqrt{N\ell} \\ &\quad + t \cdot t_2 \cdot \sigma_w \cdot \sqrt{N\ell} \\ &\quad + \nu \cdot n \cdot \hat{k} \cdot t_2^2 \cdot \sigma_{\mathbf{B}_{\mathcal{E}}} \cdot N \cdot \sigma_{\text{ctx}} \\ &\quad + n \cdot \nu \cdot q \cdot t_2 \cdot \sigma_{\text{ctx}} \cdot \sqrt{N} \\ &\quad + q \cdot \nu \cdot n \cdot \hat{\ell} \cdot t_2^2 \cdot \sigma_{\mathbf{B}_{\mathcal{E}}} \cdot N \cdot \sigma_{\text{ctx}} \\ &\quad + t \cdot t_2^2 \cdot \sigma_{\mathbf{B}_{\mathcal{E}}} \cdot N \cdot \sigma_{\text{ctx}} \cdot \hat{k} \\ &\quad + q \cdot t \cdot t_2 \cdot \sigma_{\text{ctx}} \cdot \sqrt{N} \\ &\quad + t \cdot q \cdot \hat{\ell} \cdot t_2^2 \cdot \sigma_{\mathbf{B}_{\mathcal{E}}} \cdot N \cdot \sigma_{\text{ctx}} \\ &\quad + q \cdot t \cdot t_2 \cdot \sigma_{\text{TDec}}. \end{aligned} \tag{5}$$

$\square$

C.5 Proof of Theorem 7

PROOF. We prove this through a series of hybrids written out in full detail. The simulator $\mathcal{B}$ is communicating with a challenger $\mathcal{D}$, and we show that a forgery by $\mathcal{A}$ can be used by $\mathcal{B}$ to answer $\mathcal{D}$.

G₀: The first game corresponds to the real world. Specifically, $\mathcal{B}$ follows the protocol according to the description, and $\mathcal{A}$ interacts with $\mathcal{B}$ arbitrarily. The oracles H_0, H_1 , and H_2 are simulated using tables $\mathcal{HT}_0, \mathcal{HT}_1$, and $\mathcal{HT}_2$. Since we assume a rushing adversary, $\mathcal{B}$ sends its messages first. When $\mathcal{A}$ outputs a forgery $(\Psi^* = (c^*, z^*, h^*), \mu^*)$, $\mathcal{B}$ aborts if $\mu^* \in \mathcal{M}$. If not computes $\mathbf{w}^* := \lfloor \mathbf{A}_S z^* - 2^{q_y} \cdot c^* \cdot \mathbf{y}' \rfloor_{q_w} + \mathbf{h}$. If the challenge $c^* \neq H_2(\text{pk}_S, \mathbf{w}^*, \mu^*)$, $\mathcal{B}$ aborts, otherwise halts with output (Ψ^*, μ^*) and $\mathcal{A}$ succeeds.

If the real world is indistinguishable from the programmable random oracle model, then the random oracle simulation is perfect, $\mathcal{B}$'s behavior is exactly the same as in the unforgeability experiment, and we get that

$$\Pr[\mathbf{G}_0] \leq \text{Adv}^{\text{ts-uf-cma}}(\mathcal{A}).$$

G₁: In this game $\mathcal{B}$ changes how non-interactive zero-knowledge proofs for KGen_S are answered for honest parties. $\mathcal{B}$ executes the protocol the same as $\mathbf{G}_0$, but instead of honestly generating $\pi_{s,i}$ for $i \in \mathcal{H}$, $\mathcal{B}$ uses the corresponding honest-verifier zero-knowledge simulator Sim_s for the relation $\mathcal{R}_s$ where the proof $\pi_{s,i} := \text{Sim}(\mathbf{A}_S, y_i, \text{pk}_E, \text{ctx}_{s,i}, B_s)$. $\mathcal{B}$ follows the remaining parts of the protocol honestly. $\mathbf{G}_0$ and $\mathbf{G}_1$ is indistinguishable by ZK property of the NIZK:

$$\Pr[\mathbf{G}_1] - \Pr[\mathbf{G}_0] \leq |\mathcal{H}| \cdot \epsilon_{\text{ZK},s}.$$

G₂: This is the same as $\mathbf{G}_1$ except $\mathcal{B}$ uses the extractability properties of the NIZK to learn the signing key shares in $\mathcal{C}$. During key generation, after receiving $\pi_{s,j}$ for $j \in \mathcal{C}$, $\mathcal{B}$ calls the extractor $s_j := \text{Extract}_s(\pi_{s,j}; \mathbf{A}_S, y_j, \text{pk}_E, \text{ctx}_{s,j}, B_s)$. If any of the extractions fails, $\mathcal{B}$ aborts.

By assumption, the extractor Extract_s for $\mathcal{R}_s$ is efficiently computable. If $\mathcal{B}$ does not abort due to a failed extraction, $\mathbf{G}_1$ and $\mathbf{G}_2$ is indistinguishable for $\mathcal{A}$. $\mathcal{A}$'s advantage in $\mathbf{G}_2$ is bounded by the sum of independent extraction failures:

$$\Pr[\mathbf{G}_2] - \Pr[\mathbf{G}_1] \leq |\mathcal{C}| \cdot \epsilon_{\text{extract},s}.$$

G₃: In this game $\mathcal{B}$ changes how non-interactive zero-knowledge proofs for Sign_S are answered for honest parties. $\mathcal{B}$ executes the protocol the same as $\mathbf{G}_2$, but instead of honestly generating $\pi_{r,i}$ for $i \in \mathcal{H}_U$, $\mathcal{B}$ uses the corresponding honest-verifier zero-knowledge simulator, Sim_r for relation $\mathcal{R}_r$ where the proof of boundedness $\pi_{r,i} := \text{Sim}(\mathbf{A}_S, \text{com}_i, \text{pk}_E, \text{ctx}_{r,i}, B_r)$. $\mathcal{B}$ follows the remaining parts of the protocol honestly. $\mathbf{G}_2$ and $\mathbf{G}_3$ is indistinguishable by ZK property of the NIZK:

$$\Pr[\mathbf{G}_3] - \Pr[\mathbf{G}_2] \leq |\mathcal{H}_U| \cdot \epsilon_{\text{ZK},r}.$$

G₄: $\mathbf{G}_4$ is the same as $\mathbf{G}_3$ except $\mathcal{B}$ uses the extractability properties of the Sign_S NIZK to learn the signature randomness encrypted by parties in $\mathcal{C}_U$. During signing after receiving $\pi_{r,i}$, $\mathcal{B}$ computes $\mathbf{r}_i := \text{Extract}_r(\pi_{r,i}; \mathbf{A}_S, \text{com}_i, \text{pk}_E, \text{ctx}_{r,i}, B_r)$. If any of the extractions fails, $\mathcal{B}$ aborts.

By assumption, the extractor Extract_r for $\mathcal{R}_r$ is efficiently computable. If $\mathcal{B}$ does not abort due to a failed extraction, $\mathbf{G}_3$ and $\mathbf{G}_4$ is indistinguishable for $\mathcal{A}$. $\mathcal{A}$'s advantage in $\mathbf{G}_4$ is bounded by the sum of independent extraction failures:

$$\Pr[\mathbf{G}_4] - \Pr[\mathbf{G}_3] \leq |\mathcal{C}_U| \cdot \epsilon_{\text{extract},r}.$$

G₅: $\mathbf{G}_5$ is exactly the same as $\mathbf{G}_4$ except $\mathcal{B}$ computes and sends a random hash commitment during signing for each honest party then programs the oracle a posteriori. For each $i \in \mathcal{H}_U$, after computing $\mathbf{r}_i, \mathbf{e}'_i, \mathbf{w}_i, \text{ctx}_{r,i}$, $\mathcal{B}$ samples random $h_{w,i} \in \{0, 1\}^{\ell_1}$ and broadcasts it. After receiving $h_{w,j}$ for $j \in \mathcal{C}_U$, $\mathcal{B}$ programs the oracle so that $h_{w,i} = H_1(i, \mathbf{w}_i)$ and aborts if the programming fails at any point. The rest remains the same. Since each h_i is sampled from the range of H , $\mathbf{G}_5$ is then indistinguishable from $\mathbf{G}_4$ as long as programming does not fail. Hence:

$$|\Pr[\mathbf{G}_5] - \Pr[\mathbf{G}_4]| \leq \frac{|\mathcal{H}_U|(Q_H + Q_S)}{2^{\ell_1}}.$$

G₆: In this hybrid $\mathcal{B}$ changes how ciphertexts are computed. $\mathcal{B}$ fixes some index $i' \in \mathcal{H}$ which is also in $\mathcal{H}_U$. $\mathcal{B}$ computes $\text{ctx}_{s,i'} := \text{Enc}(\text{pk}_E, 0)$ during key generation and $\text{ctx}_{r,i'} := \text{Enc}(\text{pk}_E, 0)$ during signing. The rest of the game is as it is in $\mathbf{G}_4$. Since $\mathbf{r}_j$ and $\mathbf{s}_j$ for $j \in \mathcal{C}_U$ are extracted in $\mathbf{G}_3$ and $\mathbf{G}_4$ respectively, $\mathcal{B}$ can compute $\mathbf{z} = c \sum_{i \in \mathcal{U}} \mathbf{s}_i + \sum_{i \in \mathcal{U}} \mathbf{r}_i$, which is the combined signature hence the threshold decryption can be simulated. We can now reduce to the indistinguishability of $\mathcal{E}$:

$$|\Pr[\mathbf{G}_6] - \Pr[\mathbf{G}_5]| \leq 2 \cdot \epsilon_{\text{IND}}.$$

G₇: In this hybrid $\mathcal{B}$ changes how $h_{y,i'}$ is computed. $\mathcal{B}$ sends a random $h_{y,i'} \in \{0, 1\}^{\ell_0}$ as the commitment $\mathbf{y}_{i'}$ then programs H_0 such that $H_0(i', \mathbf{y}_{i'}) = h_{y,i'}$. If programming fails, $\mathcal{B}$ aborts. Then the only difference in $\mathcal{A}$'s view is how $h_{y,i'}$ is computed. By the oracle assumption, the distributions are indistinguishable between these two games hence from the view of the adversary $\mathcal{A}$, $\mathbf{G}_6$ and $\mathbf{G}_7$ are indistinguishable as long as $\mathcal{B}$ does not abort during programming. Thus we can bound the advantage of $\mathcal{A}$ in distinguishing $\mathbf{G}_6$ and $\mathbf{G}_7$ by the failure probability of programming of H_0 :

$$|\Pr[\mathbf{G}_7] - \Pr[\mathbf{G}_6]| \leq \frac{|\mathcal{H}|(Q_H + Q_S)}{2^{\ell_0}}.$$

G₈: Now, $\mathcal{B}$ removes its reliance on the secret key $\mathbf{s}$ for signing. During key generation, $\mathcal{B}$ initializes the signing challenger $\mathcal{D}$ with the parameters of the combined signature and queries the oracle to obtain public keys, then sets $\text{crs} = \mathbf{A}_S$. Instead of honestly computing $\mathbf{y}_{i'}$, $\mathcal{B}$ derives $\mathbf{y}_{i'} = \mathbf{y}' - \sum_{j \in [n] \setminus \{i'\}} \mathbf{y}_j$ where $\mathbf{y}'$ is received from $\mathcal{D}$ and uniformly samples κ_y lower bits of $\mathbf{y}_{i'}$. The rest of key generation continues as in $\mathbf{G}_5$.

For $i \in \mathcal{H}_U \setminus \{i'\}$, signing starts as $\mathbf{G}_7$. When a signing query for μ comes, it forwards μ to $\mathcal{D}$ to obtain $(c, \mathbf{z}, \mathbf{h})$. $\mathcal{B}$ then computes $\mathbf{w} := \lfloor \mathbf{A}_S \mathbf{z} - 2^{k_y} \cdot c \cdot \mathbf{y}' \rfloor_{q_w} + \mathbf{h}$, and uses $\mathbf{z}$ for simulation to obtain ds_i . For $j \in \mathcal{U} \setminus \{i'\}$, $\mathbf{w}_j$ is computed the same. $\mathcal{B}$ then derives $\mathbf{w}_{i'} = \mathbf{w} - \sum_{j \in \mathcal{U} \setminus \{i'\}} \mathbf{w}_j$ and uniformly samples κ_w lower bits of $\mathbf{w}_{i'}$. The rest of the signing proceeds as $\mathbf{G}_7$.

Since $\mathbf{A}_S, \mathbf{y}$ received from $\mathcal{D}$ is a valid MLWE instance and the signing oracle is simulated by the noise leakage from the H-MLWE hint $\mathbf{h}$, the distribution of $\mathbf{A}_S$ therefore the high bits of $\mathbf{y}_{i'}$ in $\mathbf{G}_8$ is the same as in $\mathbf{G}_7$ by linearity of operations the derived $\mathbf{y}_{i'} = \mathbf{A}_S \mathbf{s}'_{i'} + \mathbf{e}'_{i'}$ for unknown $\mathbf{s}'_{i'}$ and $\mathbf{e}'_{i'}$. Then the distribution is indistinguishable by the MLWE assumption.

$\mathcal{B}$ now has to fix the signature and its shares in a way consistent with the first round of communication in signing. Since $\mathcal{D}$ uses the base signature scheme, it is possible to obtain consistent $(c, \mathbf{z}, \mathbf{h})$ such that $\mathbf{w} := \lfloor \mathbf{A}_S \mathbf{z} - 2^{k_y} \cdot c \cdot \mathbf{y}' \rfloor_{q_w} + \mathbf{h}$. The remaining difference

in $\mathcal{A}$'s view is the $\mathbf{w}_{i'}$, which is indistinguishable from uniform the way it is computed. An adversary distinguishing between $\mathbf{w}_{i'}$ in $\mathbf{G}_7$ and $\mathbf{G}_8$ can be used as a distinguisher for MLWE. We then have:

$$|\Pr[\mathbf{G}_8] - \Pr[\mathbf{G}_7]| \leq \epsilon_{\text{H-MLWE}}.$$

The signatures in $\mathbf{G}_8$ are independent of secret key material defined as part of the protocol and solely depend on the signature received from the challenger $\mathcal{D}$. A forgery against the $\mathcal{S}$ scheme is then a forgery against the underlying signature scheme. If $\mathcal{A}$ outputs a forgery $(c^*, z^*, h^*), \mu^*, \mathcal{B}$ can submit the same forgery to $\mathcal{D}$ as a valid forgery for the underlying signature. Hence if $\mathcal{A}$ can output a forgery at the end of $\mathbf{G}_8$, it can be used to break the uf-cma security of the underlying scheme, the advantage if $\mathcal{A}$ can then be bounded as:

$$|\Pr[\mathbf{G}_8]| \leq \text{Adv}^{\text{uf-cma}}(\mathcal{B}).$$

This concludes the proof. $\square$

C.6 Proof of Theorem 8

PROOF. Since we are in the synchronous model with authenticated broadcast, all of our arguments made in Theorem 4 are still valid for the first two conditions.

If $\mathcal{A}$ does not cause abort, then protocol proceeds as intended and by the correctness of the scheme there exists no message μ_j with signature Ψ_j^* generated through protocol where verification fails. Thus, $\mathcal{A}$ cannot win through fourth condition either.

Once again, all it remains to show is $\mathcal{A}$ cannot win through the third condition which we now show cannot be the case for any of the checks in protocol.

During $\text{KGen}_{\mathcal{S}}$, $\mathcal{A}$ cannot cheat during $\text{KGen}_{\mathcal{E}}$ without detection as long as underlying encryption scheme has security against identifiable aborts, which is by assumption true. Then, $\mathcal{A}$ can only succeed if for any $j \in C$ $h_{y,j}$ or $\pi_{s,j}$ is invalid and is not detected by $\mathcal{H}$. This can only happen if (i) j found a y_j^* such that $y_j^* \neq y_j$ $H_0(j, y_j^*) = H_0(j, y_j)$ so that $h_{j,y}$ is not the correct hash commitment to y_j or (ii) j sent $\pi_{s,j}$ without knowledge of satisfying s_j . If (i) is the case, then j found a collision for H_0 , which does not happen for all but negligible probability. If (ii) is the case however, then j can be used as a cheating prover for the underlying NIZKAoK Π_{ds} , which once again cannot happen for all but negligible probability by the knowledge soundness assumption.

For $\text{Sign}_{\mathcal{S}}$, if j sends an incorrect message without any of the honest users aborting (i) j found a $\mathbf{w}_j^*$ such that $\mathbf{w}_j^* \neq \mathbf{w}_j$ $H_1(j, \mathbf{w}_j^*) = H_1(j, \mathbf{w}_j)$ so that $h_{j,y}$ is not the correct hash commitment to y_j or (ii) j sent $\pi_{r,j}$ without knowledge of satisfying r_j (iii) j sends a decryption share ds_j such that combined signature $\mathbf{z} = \perp$. If (i) is the case, then j found a collision for H_1 , which does not happen for all but negligible probability. If (ii) is the case however, then j can be used as a cheating prover for the underlying NIZKAoK Π_{ds} , which once again cannot happen for all but negligible probability by the knowledge soundness assumption. And finally if (iii) is the case either j found an incorrect share that does not combine correctly. By the correctness of $\mathcal{E}$, this can only happen if the sent ds_j is incorrectly computed which then would violate the knowledge soundness.

Since $\mathcal{A}$ cannot make any $j \in C$ cheat without detection with non-negligible probability, it cannot succeed in the identifiable aborts game. This concludes the proof. $\square$

D Zero-Knowledge Proofs

VSS Proof. In the VSS we wish to prove the the Hadamard product of the n -tuple of committed shares with a (pseudo) randomly generated element $c^\perp$ of the dual code is zero.

$$\mathcal{R}_{\text{VSS}} := \left\{ (X, W) \mid \begin{array}{l} X := (\text{pk}_{\text{Com}}, \{\text{com}_{[S]_i}\}_{i \in [n]}, C^\perp), W := (\{[S]_i, r_{[S]_i}\}_{i \in [n]}): \\ \text{for all } i \in [n]: \text{BDLOP.Open}(\text{pk}_{\text{Com}}, \text{com}_{[S]_i}, d_i := ([S]_i, r_{[S]_i}, 1) = 1) \\ \wedge \sum_{i=1}^n \Phi(S_i) \circ \Phi(C^\perp) = 0^{N \cdot (m_1 \times m_2)} \end{array} \right\} \quad (6)$$

Now that we have a way to verify sharing of an element, we define the relations to be proven for the $\mathcal{E}$ and $\mathcal{S}$ schemes. These relations are similar to ones described in Section 5 but more explicit with the help of detailed relations above. First we start with the proof for language $\mathcal{R}_{\text{KGen}_{\mathcal{E}}}$ which shows key shares are generated correctly.

$$\mathcal{R}_{\text{KGen}_{\mathcal{E}}} := \left\{ (X, W) \mid \begin{array}{l} X := (\mathbf{B}_i, \{\mathbf{B}_{i,j}\}, \{\text{com}_{i,j}\}_{i \in [n]}), \\ W := (S_i, \rho_i, E_i, \rho'_i, \{S_{i,j}\}_{j \in [n]}, \{\rho_{i,j}\}_{j \in [n]}, \{E_{i,j}\}_{j \in [n]}): \\ \|S_i\|_\infty, \|E_i\|_\infty \leq t_\infty \cdot \sigma_{\text{LWE}} \wedge \mathbf{B}_i = \mathbf{A}_{\mathcal{E}} S_i + p E_i \\ \wedge \forall j \mathbf{B}_{i,j} = \mathbf{A}_{\mathcal{E}} S_{i,j} + p E_{i,j} \wedge \mathbf{B}_j = \text{WSS.Rec}_{t,n}(\{\mathbf{B}_{i,j}\}) \\ \wedge S_i = \text{WSS.Rec}_{t,n}(\{S_{i,j}\}) \wedge E_i = \text{WSS.Rec}_{t,n}(\{E_{i,j}\}) \\ \forall i \text{BDLOP.Open}(\text{com}_{i,j}, S_{i,j}, \rho_{i,j}) = 1 \end{array} \right\} \quad (7)$$

Next, we show the relation for correct partial decryption.

$$\mathcal{R}_{\text{ds}} := \left\{ (X, W) \mid \begin{array}{l} X := (\text{ds}_i, \text{ctx}, \text{com}_{\text{sk},i}, \text{com}_{E,i}), W := (\text{sk}_i, \rho_{\text{sk},i}, E_i, \rho_{E,i}): \\ \|E_i\|_\infty \leq t_\infty \cdot \sigma_{\text{TD}} \wedge \text{ds}_i = \lambda_i \text{sk}_i \mathbf{u} + p E_i \wedge \\ \text{BDLOP.Open}(\text{com}_{\text{sk},i}, \text{sk}_i, \rho_{\text{sk},i}) = 1 \\ \wedge \text{BDLOP.Open}(\text{com}_{E,i}, E_i, \rho_{E,i}) = 1 \end{array} \right\} \quad (8)$$

The commitments are implicit in the proof system used similar to relations for our WSS. Finally we define the two relations in $\mathcal{S}$ itself which contain two similar proofs of encryption showing knowledge of a relevant plaintext. Since the commitment to the secret key shares are also included as part of the distributed key generation, it is trivial to prove the above statements using the proof of linear relations as before, once the commitments to E_i are available. For which we instantiate the proof $\pi_{\text{ds},i}$ as a proof of linear relation to show d is correctly computed alongside a commitment to E_i .

$$\mathcal{R}_{\text{S}} := \left\{ (X, W) \mid \begin{array}{l} X := (\mathbf{A}_{\mathcal{S}}, y_i, \text{pk}_{\mathcal{E}}, \text{ctx}_{S_i}), W := (s_i, e_i): \\ \|s_i\|_\infty, \|e_i\|_\infty \leq t_\infty \cdot \sigma_{\mathcal{S}} \wedge y_i = \mathbf{A}_{\mathcal{S}} s_i + e_i \\ \wedge \text{ctx}_{S_i} = \text{Enc}(\text{pk}_{\mathcal{E}}, s_i) \end{array} \right\} \quad (9)$$

$$\mathcal{R}_{\text{r}} := \left\{ (X, W) \mid \begin{array}{l} X := (m_{\mathcal{A}}, \text{pk}_{\mathcal{E}}, \text{ctx}_{r_i}), W := (w_i, r_i): \\ \|r_i\|_\infty, \|e'_i\|_\infty \leq \sigma_{\text{r}} \wedge \text{ctx}_{r,i} = \text{Enc}(\text{pk}_{\mathcal{E}}, r_i) \\ \wedge w_i = \mathbf{A}_{\mathcal{S}} r_i + \wedge \text{com}_i = \text{BDLOP.Com}(w_i, \rho_i) \end{array} \right\} \quad (10)$$

Both of these languages need a proof of plaintext knowledge which we define below:

$$\mathcal{R}_{\text{Enc}} := \left\{ (X, W) \mid \begin{array}{l} X := (\text{ctx} = (\mathbf{u}, \mathbf{v}), \text{com}_{\text{r}}, \text{com}_{e_1}, \text{com}_{e_2}, \text{com}_{\mathbf{m}}), \\ W := (\mathbf{r}, e_1, e_2, \mathbf{m}, \rho_{\text{r}}, \rho_{e_1}, \rho_{e_2}, \rho_{\mathbf{m}}): \\ \|r\|_\infty, \|e'_1\|_\infty, \|e''_2\|_\infty \leq B_{\text{Enc}} \wedge \mathbf{u} = \mathbf{A}_{\mathcal{E}}^T \mathbf{r} + p e_1 \\ \wedge \mathbf{v} = \mathbf{B}_{\mathcal{E}}^T \mathbf{r} + p e_2 + \mathbf{m} \wedge \|\mathbf{m}\|_\infty \leq p \\ \wedge \text{BDLOP.Open}(\text{com}_{\text{r}}, \mathbf{r}, \rho_{\text{r}}) = 1 \\ \wedge \text{BDLOP.Open}(\text{com}_{e_1}, e_1, \rho_{e_1}) = 1 \\ \wedge \text{BDLOP.Open}(\text{com}_{e_2}, e_2, \rho_{e_2}) = 1 \\ \wedge \text{BDLOP.Open}(\text{com}_{\mathbf{m}}, \mathbf{m}, \rho_{\mathbf{m}}) = 1 \end{array} \right\} \quad (11)$$

Commitments are once again implicit to the proof system.